Automation in Construction 186 (2026) 106888

Contents lists available at ScienceDirect

Automation in Construction

journal homepage: www.elsevier.com/locate/autcon

Multi-agent framework for schema-guided reasoning and tool-augmented interaction with IFC models

Yan Gao [a], Fuji Hu [a], Chengzhang Chai [b], Yiwei Weng [a,*], Haijiang Li [b]

[a] Department of Building and Real Estate, The Hong Kong Polytechnic University, Hong Kong, China
[b] BIM for Smart Engineering Centre, School of Engineering, Cardiff University, Cardiff CF24 3AA, UK

ARTICLE INFO

Keywords:
IFC querying
LLM agents
Multi-agent systems
Building information modelling (BIM)
Tool-augmented reasoning

ABSTRACT

Despite recent advances in large language model (LLM)-based agents, their applications to schema-guided reasoning and interaction with Building Information Modelling (BIM) data remain limited. This paper presents IFC-Agent, a tool-augmented multi-agent framework that enables natural language querying, reasoning, and modification of Industry Foundation Classes (IFC) models. The framework integrates schema-guided traversal with LLM-driven dynamic tool composition to achieve interpretable and scalable reasoning workflows. An adaptive dual-mode execution strategy combines asynchronous parallel execution with exploration–aggregation reasoning, while a dual-memory mechanism ensures operational efficiency and semantic consistency. A prototype system was developed using LangChain and validated on benchmark IFC queries. Case studies demonstrate strong performance in field querying, multi-hop reasoning, coordinate transformation, and IFC graph construction. Comparative evaluation with recent graph-based and LLM-assisted BIM frameworks highlights IFC-Agent's advantages in schema-guided reasoning and natural language interaction, establishing a practical foundation for explainable and automated BIM workflows.

1. Introduction

Building Information Modelling (BIM) has become an indispensable technology in the Architecture, Engineering, and Construction (AEC) industry, which provides a unified digital representation of building assets across disciplines and life-cycle stages. The Industry Foundation Classes (IFC) standard enables open and interoperable data exchange among heterogeneous platforms but interacting with IFC models remains challenging due to their complex schema hierarchies and dense relational structures [1]. Existing BIM authoring tools and application programming interfaces (APIs) primarily rely on fixed query commands or procedural scripts, which require users to understand complex schema hierarchies and data dependencies [2,3]. These systems offer deterministic but rigid access to model information, making it difficult to support flexible, natural-language-driven queries and multi-hop reasoning, or fine-grained model modification. As a result, both experts and non-experts encounter difficulties when exploring or modifying IFC data beyond predefined workflows [4]. These limitations highlight the need for more intuitive and schema-aware mechanisms for natural language interaction with BIM models.

Recent advances in Large Language Models (LLMs) have provided a promising way to address these challenges. LLMs exhibit strong capabilities in semantic understanding and contextual reasoning, along with emergent abilities that enable generalization beyond training data [5]. These capabilities allow them to interpret and manipulate structured data through natural language instructions and tool-augmented reasoning. In the BIM domain, prior studies [6–9] have explored LLMs for property extraction, semantic enrichment, and BIM element generation. However, most existing approaches [4,10,11] rely on isolated prompting, retrieval-augmented generation (RAG), or model fine-tuning, which, while effective for textual reasoning, are insufficient for schema-guided execution on complex IFC structures. These limitations reveal a critical research gap: how to enable scalable, schema-guided, and tool-augmented reasoning on native IFC data through an LLM-based agentic workflow.

To address this gap, this paper proposes IFC-Agent, a modular, schema-guided, and tool-augmented multi-agent framework that enables LLMs to perform interpretable reasoning and execution directly on IFC data. The framework adopts a hierarchical multi-agent architecture and employs an adaptive dual-mode execution strategy that dynamically selects the reasoning route according to the task scale (i.e., number of target entities). For small-scale tasks, the system performs asynchronous

* Corresponding author.
E-mail address: yiwei.weng@polyu.edu.hk (Y. Weng).

https://doi.org/10.1016/j.autcon.2026.106888
Received 27 October 2025; Received in revised form 8 March 2026; Accepted 11 March 2026
Available online 23 March 2026
0926-5805/© 2026 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

execution, with multiple sub-agents running in parallel to process individual entities independently. For large-scale tasks, the system activates the exploration-aggregation mode, which operates under the assumption of structural homogeneity, i.e., the entities of the same type share identical structural hierarchies and field definitions. In this mode, the agent explores a few sampled entities to infer reusable toolchains and generalizes them to all remaining entities, which significantly improves execution efficiency while avoiding token explosion and redundant LLM calls. Each agent functions within a schema-guided workflow, where tool selection and chaining are dynamically inferred from the IFC schema rather than pre-defined templates. Moreover, the framework integrates a dual-memory design, combining a *CustomMemory* module that records procedural traces and reasoning logs with structured semantic caches (*globalid_dict* and *stepid_dict*) that store parsed IFC entities. This design ensures consistent cross-agent reuse and enables the IFC-Agent to gradually expand its understanding of the model through consecutive queries.

Moreover, a prototype system was developed to implement the IFC-Agent framework using LangChain [12] and IfcOpenShell [13], which integrates LLM-based semantic reasoning with a suite of atomic, schema-aligned tools. These tools provide flexible, fine-grained operations that can be dynamically composed by the LLM to perform schema-guided reasoning. The system was validated on a benchmark IFC query set constructed following the taxonomy of prior studies [2,3] for BIM query, and evaluated across three OpenAI models (GPT-3.5-turbo, GPT-4o-mini, and GPT-4o) under identical configurations. GPT-4o successfully completed all 48 benchmark tasks, demonstrating superior capabilities in multi-hop reasoning, execution efficiency, and robustness. Further case studies verified the framework's effectiveness in (i) schema-guided field querying and modification, (ii) task decomposition and exploration-aggregation execution, (iii) coordinate transformation through intrinsic matrix computation, and (iv) incremental IFC-graph construction. In addition, a qualitative comparison was conducted against prior representative approaches, including ifcOWL [14,15] and GQL4BIM [2,3], highlighting the IFC-Agent's superior flexibility, interpretability, and scalability in schema-guided reasoning and natural language interaction. These results confirm the framework's effectiveness and scalability, as well as demonstrate its potential as a practical foundation for explainable, schema-aware, and tool-augmented interaction with IFC-based BIM models.

The main contributions of this paper are as follows:

1) A schema-guided, tool-augmented multi-agent framework that enables interpretable reasoning on native IFC data through dynamic tool composition, without requiring static graph transformations or predefined procedural templates.
2) An adaptive dual-mode execution strategy that addresses the interpretability-scalability trade-off in schema-guided reasoning. The framework switches between asynchronous execution for heterogeneous small-scale tasks and exploration-aggregation for homogeneous large-scale queries based on execution feasibility, ensuring robust reasoning across diverse query scenarios.
3) A dual-memory architecture integrating procedural context (*CustomMemory*) with entity-level semantic caches (*globalid_dict* and *stepid_dict*) that enables cross-agent reuse, incremental model understanding, and consistent reasoning across tasks without redundant parsing.
4) Experiments demonstrate that LLMs can leverage implicit schema awareness to perform autonomous multi-hop traversal of IFC structures using atomic, lightweight tools without rigid preprocessing, with reasoning performance systematically enhanced through schema-aligned prompt constraints. The experiments further reveal that tool augmentation extends the agent's execution capabilities but cannot replace the LLM's intrinsic reasoning capacity.

The rest of this paper is structured as follows: Section 2 provides an overview of related work on LLM-based applications for BIM and existing methods for interacting with IFC data. Section 3 presents the proposed framework and system architecture in detail. Sections 4 and 5 cover the validation of the framework through benchmark experiments and representative case studies, respectively. Finally, Sections 6 and 7 discuss the findings and conclude the paper with future research directions.

2. Related work

This section reviews LLM-based approaches for BIM tasks and existing techniques for IFC data access and querying. Section 2.1 covers model adaptation strategies and multi-agent architectures; Section 2.2 reviews traditional, graph-based, and LLM-driven IFC querying techniques; and Section 2.3 identifies the key research gaps.

2.1. LLM-based applications for BIM

As a core component of the AEC industry, BIM poses unique challenges for natural language querying, semantic reasoning, and intelligent model manipulation, due to its deeply nested references, schema-defined relational chains, multi-entity scale, and the need to support both information retrieval and model modification. Recent progress in LLMs has made it possible to interact with BIM data and model in more flexible and intelligent ways, which requires both complex structural and semantic reasoning. Building on these advances, prior studies [6–8,16–18] have explored three mainstream approaches for adapting general-purpose LLMs to BIM domain, including prompt engineering, retrieval-augmented generation (RAG), and model fine-tuning. Each of them offers different trade-offs in scalability, task alignment, and implementation complexity. Moreover, beyond adapting individual models, current studies [19–22] have increasingly focused on advanced system architectures, such as multi-agent frameworks and tool-augmented pipelines, which combine prompting, RAG, and fine-tuning in complementary ways, aiming to support more complex and diverse tasks beyond the scope of basic model adaptation.

2.1.1. Adapting LLMs to BIM tasks

Prompt engineering, RAG, and fine-tuning represent three major approaches for adapting general-purpose LLMs to BIM tasks. These approaches are not mutually exclusive but complementary: prompting provides flexibility, RAG improves factual grounding by incorporating external knowledge, and fine-tuning delivers higher precision for specialized or repetitive tasks. Each comes with its own trade-offs in scalability, adaptability, and implementation complexity, and in practice they can be combined within unified frameworks or systems to address complex tasks, such as multi-agent BIM model generation [6] and cross-domain integration with energy simulation [8]. Representative studies for each approach are outlined below.

Prompting leverages carefully designed instructions or in-context examples to guide pre-trained LLMs without changing model weights. In the BIM domain, prompting has been applied to tasks such as information extraction, property identification, and IFC element generation. For example, ChatBIM [16] uses in-context prompts to retrieve and visualize IFC data, while Text2BIM [6,7] adopts prompt chaining in a multi-agent setup to generate IFC elements. These studies demonstrate that, with well-structured prompts, general-purpose LLMs (e.g., GPT-4o, Claude-4) can follow schema-related cues in prompts to some extent, though their capacity remains limited for more complex or fine-grained BIM tasks. Meanwhile, prompting alone is constrained by context length and IFC's deeply nested structure, making it insufficient for multi-hop reasoning or fine-grained model editing without external support.

RAG extends LLM capabilities by grounding responses in external data sources, such as schema fragments, design standards, or semantic subgraphs. This approach improves factual accuracy and reduces hallucination by dynamically injecting relevant information into the

model's context window. For instance, Iranmanesh et al. [10] proposed a graph-RAG pipeline that transforms IFC models into a Neo4j property graph and answers queries by executing LLM-generated Cypher codes, while Du et al. [6] incorporated RAG in a BIM copilot to retrieve external design standards as contextual knowledge. These studies highlight RAG's strength in context-aware reasoning, especially when domain knowledge is too large to fit within prompt limits. However, its effectiveness depends on retrieval quality and indexing strategies, which remain challenging for IFC's complex semantics.

Fine-tuning adapts foundation models with BIM-specific datasets, aiming to produce more structured and accurate outputs. For example, Zhang et al. [8] fine-tuned an LLM to translate textual building descriptions into structured energy models, Forth and Borrmann [18] trained multilingual models to semantically enrich BIM elements, and Du et al. [17,23] fine-tuned a transformer on workflow logs to predict next-step authoring commands in BIM tools. Fine-tuning can achieve high fidelity and context-aware outputs, making it well suited to specialized or repetitive tasks. However, it typically requires large-scale annotated datasets and heavy computational resources for training. Moreover, fine-tuned models often generalize poorly across different schema variants or software environments. They may also suffer from catastrophic forgetting when transferred beyond their training distribution, thereby losing some of the general-purpose capabilities of the original foundation models.

2.1.2. Multi-agent systems for BIM

The growing demand for natural language interaction with BIM models has motivated the development of LLM-based systems tailored to this domain. Unlike general NLP tasks, BIM requires schema-aware execution for property assignment, spatial reasoning, and multi-hop traversal across deeply nested and interrelated entities. While prompting, RAG, and fine-tuning have demonstrated strengths in language understanding and knowledge retrieval, they remain limited when applied directly to parsing and manipulating complex BIM models at scale. As a result, recent studies [19–22] have explored multi-agent frameworks combined with tool augmentation to improve modularity, scalability, and task coverage. Existing LLM-based multi-agent architectures can be broadly categorized into sequential pipelines [24], hierarchical planner-executor designs [25], and hybrid approaches [6,7]. Within these systems, tool augmentation plays a central role by coupling LLM reasoning with external functions, including hard-coded APIs for deterministic operations [9], LLM-generated code for flexible execution [10], and prompt- or model-based soft tools for loosely defined tasks [6,7]. In practice, agents translate natural language queries into reasoning steps and coordinate tool invocation, while tools serve as execution modules providing deterministic, heuristic, or code-level operations.

Recent studies have demonstrated the effectiveness of such frameworks for BIM-related tasks. Zhang et al. [8] proposed a hybrid multi-agent system for building energy analysis, integrating hierarchical coordination, tool augmentation, and memory modules. Du et al. [6,7] developed the Text2BIM system, where role-specific agents collaboratively generate BIM models from free-form instructions. Guo et al. [9] mapped natural language queries to Revit C# APIs using keyword-task routing and constraint alignment, achieving promising results for information retrieval. However, these approaches remain largely confined to specific platforms (e.g., Revit) or focus primarily on information access, with limited support for schema-guided reasoning directly over native IFC data, particularly for multi-hop and computation-intensive tasks.

These multi-agent BIM systems [6–10] exhibit two recurring design principles: memory management and execution orchestration. Memory management, such as dual-memory for conversational context and task states, workflow-level state tracking, and memory filtering with supervisory guidance, is used to support coherence, reuse intermediate results, and reduce hallucinations in multi-agent collaboration. Execution orchestration, including sequential or parallel scheduling as well as synchronous or asynchronous execution, affects system throughput and robustness when handling multiple entities and tool interactions. However, these systems mainly differ in orchestration strategies, rather than in their underlying reasoning paradigms. Although asynchronous execution improves efficiency and fault tolerance, it remains constrained by API rate limits, computational resources, and LLM context capacity. More critically, these systems do not explicitly distinguish between information-oriented queries and execution-oriented tasks that require schema-constrained, multi-hop computation with verifiable intermediate steps. As a result, large-scale or computation-intensive BIM tasks remain difficult to execute reliably.

2.2. IFC data access and querying techniques

Accessing information in IFC models is inherently challenging because of their hierarchical schema, nested references, and dense inter-entity relationships. Early research [14,15,26,27] attempted to overcome these issues through procedural parsing and format transformation, which enabled basic interoperability but offered limited semantic reasoning. Later efforts [2,3] turned to graph-based representations and ontology approaches, aiming to capture IFC's relational structure for more flexible semantic queries. Most recently, with the advent of LLMs, IFC data access has been extended toward natural language interfaces and agentic workflows [4,10,28]. The following subsections review these developments, highlighting how the field has progressed from procedural to graph-based, and now to LLM-driven approaches.

2.2.1. Traditional processing and transformation approaches

Early approaches for IFC data processing often relied on converting IFC models into other structured formats, such as OWL [15] and XML [26,27], to enable data access and querying within external systems. While these transformations support basic interoperability, they typically preserve only partial building semantics and lose many domain-specific details during the conversion process. To address these limitations, a wide variety of software libraries and platforms have been developed to support IFC data processing. They span multiple programming languages and use cases, from lightweight scripting to large-scale BIM model management. Examples include IfcOpenShell (C++/Python) [29], the most widely used library for low-level IFC parsing and geometry reconstruction; IfcPlusPlus [30], a C++ toolkit featuring built-in geometry handling and Qt-based GUI; ifc.js [31], a modular, browser-based framework that includes components like web-ifc (for reading/writing IFC files), web-ifc-three, and web-ifc-viewer for 3D visualization; and BIMServer [32], a Java-based platform supporting multi-user collaboration, version control, and incremental model updates. In addition to standalone toolkits, many commercial BIM authoring platforms offer plugin-based support for IFC model import, export and limited manipulation, including Revit (with the official IFC Exporter), Navisworks, Tekla Structures, ArchiCAD, and Vectorworks. These tools facilitate IFC interoperability within traditional design workflows. Open-source modelling and visualization tools like BlenderBIM [33] and FreeCAD [34] also support IFC integration, typically via IfcOpenShell or custom parsers. These platforms enable users to view, edit, and convert IFC models in non-proprietary environments. For example, BlenderBIM allows for the direct authoring of IFC elements within a parametric 3D modelling framework.

Despite these capabilities, most existing solutions are built upon procedural parsing and direct field access, lacking support for semantic-level reasoning, relationship tracing, or flexible multi-hop queries across interconnected elements. As a result, these approaches require substantial expertise in the IFC schema and offer limited automation and scalability for advanced reasoning tasks [35]. This gap has motivated recent research into alternative representations and intelligent assistants, such as graph-based semantic query frameworks [2,3] and LLM-

driven agentic workflows [4,10], which seek to overcome the limitations of purely procedural IFC processing.

2.2.2. Graph representations and semantic querying

To address the limitations of procedural parsing, researchers have explored graph-based representations of IFC data. A representative effort is the IFC-Graph [2], which converts IFC entities into a labeled property graph (LPG) stored in Neo4j to enable semantic querying via Cypher. In this framework, key structural relationships, such as spatial containment, connectivity, and material associations, are represented as graph edges, allowing multi-hop reasoning across interconnected entities. Based on this work, GQL4BIM [3] introduces semantic path abstraction, which defines reusable query templates for commonly used relationships (e.g., element-material links or space-boundary-element chains). These paths are grounded in both the IFC schema and the graph topology, enabling users to formulate high-level semantic queries without manually navigating low-level schema hierarchies. Meanwhile, earlier work on ifcOWL and related RDF/ontology approaches [15,36] followed a similar idea by exposing IFC semantics through SPARQL querying. These ontology-based methods explicitly model IFC concepts and relationships as triples, which can be directly interpreted as graph structures as well. Recent studies have also explored graph-based representations to enhance semantic reasoning over infrastructure data, such as a knowledge-graph framework [37] that leverages text encoding and graph neural networks (GNNs) to organize maintenance information into a structured ontology graph for relational inference and natural language-based querying. However, such approaches still depend on fixed schema traversal and graph query languages (e.g., Cypher or SPARQL), which primarily support representation-level semantic exploration.

While graph-based approaches improve interoperability and enable multi-hop semantic traversal, they face several inherent limitations when applied to execution-oriented BIM tasks. Firstly, while graph-based approaches can effectively retain IFC's hierarchical and referential structures, they require heavy preprocessing to convert the entire IFC model into graph representations upfront, before any querying begins. This preprocessing overhead and reliance on predefined transformation pipelines restrict flexibility for dynamic reasoning, incremental execution, and real-time interaction. Secondly, such approaches require heavy preprocessing and rely on predefined templates or transformation pipelines, which restrict flexibility for dynamic reasoning, incremental execution, and real-time interaction. As a result, most graph-based frameworks remain focused on information retrieval and relational querying, rather than accountable execution involving schema-constrained computation and aggregation across entities. These limitations highlight the need for approaches that can operate directly on native IFC files and support schema-guided, context-aware reasoning during task execution, without relying on intermediate graph transformations.

2.2.3. LLM-based query assistants

Although general-purpose LLMs such as ChatGPT have shown the ability to interpret elements of the IFC schema and parse short IFC code fragments or STEP expressions, research on LLM-based query assistants for IFC querying and reasoning remains limited. One of the earliest explorations is IFC Insight [28], a prototype built on ChatGPT that enables users to query IFC concepts, inspect entity relationships, and generate procedural scripts (e.g., Python for IfcOpenShell or JavaScript for ifc.js) via natural language. This lowers the entry barrier for non-experts, but the system cannot directly operate on IFC files, lacks memory to maintain task context, and provides little support for schema-guided navigation of relational fields. It also depends on isolated prompts and LLM-generated code without mechanisms for verification or reproducibility. As a result, multi-step queries, fine-grained modifications, and consistent model-level reasoning remain difficult to achieve.

To address these issues, recent studies have begun to integrate LLMs with external tools and structured representations. Hellin et al. [4] developed an LLM-based agentic workflow that integrates chain-of-thought prompting with the ReAct paradigm to execute Python functions on IFC models. This design demonstrates the ability to handle certain multi-hop queries more effectively than isolated prompting, as sequential tool invocation provides a form of structured reasoning and partial reproducibility. In parallel, Iranmanesh et al. [10] proposed a Graph-RAG pipeline that grounds LLM outputs in explicit IFC subgraphs, converting models into graph representations and generating Cypher queries with GPT-4o. This approach reduces hallucinations and improves factual grounding by anchoring responses to retrieved graph fragments. While these studies demonstrate clear progress, they rely on intermediate representations (e.g., extracted IFC graphs) and predefined transformation pipelines, rather than operating directly on native STEP-level entities during query execution, which limits adaptability to varied query types. Moreover, in these systems, the LLM primarily acts as a semantic interpreter or code generator, producing either textual answers or one-shot scripts. The correctness of intermediate execution steps is not explicitly modelled, and errors cannot be systematically traced or constrained during reasoning. Therefore, these approaches remain focused on information access rather than accountable execution over native IFC data.

2.3. Summary and research gaps

Existing BIM querying and information access approaches have progressed along two main directions. On the one hand, graph-based and ontology-driven methods transform IFC models into alternative representations to enable semantic querying and multi-hop traversal across entities. On the other hand, recent LLM-based systems introduce natural language interfaces, retrieval-augmented reasoning, or agentic workflows to improve usability and accessibility. As reviewed in Sections 2.1 and 2.2, these approaches are effective for information retrieval and semantic exploration, particularly when operating on abstracted representations such as graphs, knowledge bases, or retrieved textual contexts. However, when applied to native IFC-centric tasks, several limitations remain:

- Firstly, most graph-based and Graph-RAG approaches rely on extensive preprocessing and intermediate representations, in which IFC semantics are re-encoded into node-edge graph structures or predefined query templates. While this enables efficient semantic querying, the transformation from IFC to graph may introduce semantic deviations, and schema knowledge needs to be resolved and fixed in advance. As a result, traversal logic along schema-defined paths is determined by pre-constructed graph structures or templates, limiting the ability to adapt traversal depth, access paths, or execution strategies to query-specific reasoning needs.
- Secondly, existing LLM-based BIM/IFC assistants primarily realize reasoning through text generation or one-shot code invocation. In such systems, the LLM produces a final answer or complete script in a single pass, without explicitly modelling intermediate reasoning states or enforcing constraints across stepwise execution. Consequently, these approaches do not support exploration-driven execution, where partial, schema-resolved results can be inspected and used to guide subsequent reasoning steps.
- More fundamentally, most existing approaches address BIM-related natural language queries primarily at the level of information retrieval or explanation, so that approximate or descriptive answers are considered acceptable. However, many practical BIM tasks require execution-grounded reasoning, where answers must be derived through explicit schema-constrained operations. Meanwhile, intermediate states must remain consistent across reasoning steps, and final results must be computationally verifiable.

These gaps do not stem from limitations in natural language

understanding or information retrieval alone, but from the absence of reasoning paradigms that treat BIM queries as execution problems rather than retrieval problems. Addressing these issues requires approaches that can autonomously determine valid actions, select and invoke appropriate tools, and iteratively explore model structures under schema constraints during task execution. Such approaches resemble how human experts reason with IFC data and motivate LLM-based agentic systems that unify querying and modification within a single execution-grounded pipeline operating directly on native IFC data.

3. Proposed method

While recent LLM-based multi-agent BIM systems have shown promising capabilities for semantic querying and tool-augmented execution, they remain constrained by several fundamental issues in real-world applications: 1) the deeply hierarchical and referential structure of the IFC schema makes reasoning complexity and token consumption grow proportionally with schema (field) depth, even for a single entity; 2) the large number of entities in practical IFC models similarly increases reasoning complexity and contextual overload proportionally; 3) the existing frameworks rely on fixed toolchains or static graph transformations. The first two challenges push systems against hardware limits, API rate limits, and LLMs' concurrency caps, and they also produce huge numbers of fragmented, redundant outputs, which leads to token explosion and LLM hallucination. The third one constrains adaptability and scaling across diverse query types.

These issues underscore the need for a schema-guided, context-aware reasoning framework coupled with an adaptive hybrid execution strategy, which can handle IFC's structural complexity against the processing limits of LLMs. This aspect has not yet been systematically explored in prior studies. To address these challenges, our method incorporates three synergistic methodological strategies:

3.1. Adaptive hybrid execution

To mitigate token explosion and maintain scalability, the system dynamically switches between asynchronous and bulk execution modes. When the target entities (e.g., GlobalIds) are only a few, agents perform fine-grained schema-aware reasoning in parallel. When their number exceeds a predefined threshold, the system enters an exploration-aggregation mode, i.e., a few of entities are sampled for toolchain exploration, and the inferred reasoning pattern (i.e., toolchains) is then applied in batch for aggregation and summarization. This approach assumes a schema-homogeneous context in the BIM model, i.e., the sampled and remaining entities share the same field hierarchies and attribute definitions within the identical schema depth, so that the inferred reasoning chain remains valid and can generalizable across the others.

3.2. Schema-aware parsing

To enable flexible and fine-grained reasoning, the framework adopts a schema-aware parsing strategy that allows each tool invocation to follow the logical structure of the IFC schema. This schema-driven approach ensures that reasoning remains modular, adaptive, and context-sensitive, allowing the system to generalize across different IFC schema versions, modelling conventions, and query types. It gives agents the flexibility to interpret and extract relevant information directly based on schema semantics, without requiring rigid predefined preprocessing pipelines. Meanwhile, this on-demand strategy also avoids the need for complete conversion of BIM models into graphs or ontologies.

3.2.1. Dual-memory coordination

To ensure coherence and efficiency, the framework separates conversational memory (for task history and intent) from structured semantic caches (i.e., *globalid_dict* and *stepid_dict)*, which store parsed IFC entities for reuse across agents. This design enables consistent reasoning, avoids redundant parsing, and supports the incremental construction of the IFC graph during execution.

To integrate these strategies into a unified architecture, we proposed IFC-Agent, a hierarchical multi-agent framework for explainable, scalable, and schema-aware interaction with IFC data. The main agent decomposes complex IFC queries into subtasks, which are executed through specialized sub-agents supported by dual memory and adaptive hybrid execution. Each sub-agent interprets schema-level relationships, performs tool-augmented operations, and collaborates within an adaptive, structured workflow that dynamically adjusts its execution mode according to task scale. Essentially, the IFC-Agent balances reasoning granularity and token load through adaptive execution, enabling LLMs to perform efficient multi-hop reasoning on complex IFC models without heavy preprocessing or rigid toolchains. The following sections detail its overall architecture, execution workflow, tool integration and dual-memory design.

3.3. Overall architecture of the system

The proposed IFC-Agent adopts an adaptive dual-mode architecture that dynamically selects its reasoning workflow based on execution feasibility, as illustrated in Fig. 1. The system routes execution through two complementary modes by considering both the scale of targeted entities and the regularity of data structural traversal, rather than relying on the fixed or static hierarchical control structure in prior studies [7–9]. Specifically, execution feasibility reflects both engineering constraints, such as context length, aggregation reliability, and computational overhead, and design considerations related to data structural regularity. For example, when tasks involve a small number of entities or potentially heterogeneous schema structures, the system favours fine-grained parallel execution to preserve per-entity reasoning fidelity. Conversely, when tasks involve a large number of entities that share homogeneous schema structures, the system will activate an exploration-aggregation mode to enable scalable execution through reusable toolchains and controlled aggregation. This adaptive routing mechanism enables IFC-Agent to balance interpretability and scalability, ensuring robust reasoning for schema-diverse tasks while maintaining efficiency for large-scale, structurally consistent IFC queries.

The main agent acts as the central controller responsible for task understanding, decomposition, and routing decisions. When receiving a natural language query, it first consults the conversational memory to check for reusable information from prior queries, thereby avoiding redundant execution. Then, it decomposes the query into multiple subtasks and performs semantic parsing to identify the relevant IFC entity types and targeted fields for each task. Although execution feasibility is influenced by factors such as scalability, aggregation reliability, and data structural regularity, the current implementation adopts entity scale as the principal criterion for mode selection. Accordingly, the main agent determines the execution mode based on the number of targeted entities identified during task parsing. This threshold is empirically calibrated through pilot executions, beyond which per-entity parallel reasoning in Mode I becomes increasingly unstable due to the accumulation of intermediate observations, as well as the limited context and aggregation capacity of the underlying LLM.

3.3.1. Mode I – asynchronous parallel execution

When the task involves a limited number of target entities, the main agent dispatches subtasks to multiple asynchronous executors, each responsible for a specific semantic domain such as geometry, quantity, property, or relation. These executors launch parallel sub-agents that perform schema-aware reasoning on individual entities using a set of fine-grained, schema-aligned tools designed to extract, traverse, or modify specific IFC fields. Each tool operates at the field level and aligns directly with the IFC schema hierarchy, allowing agents to construct

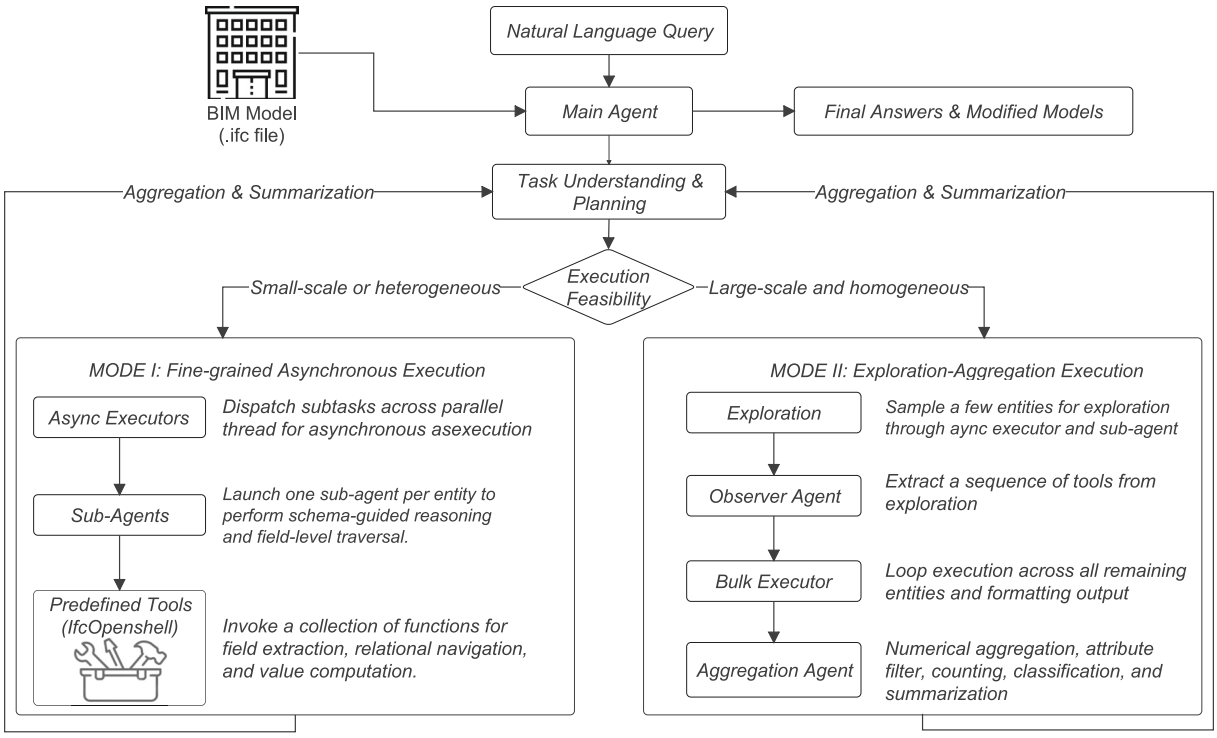


Fig. 1. Proposed IFC agent framework and system architecture.

interpretable reasoning chains step by step, maintaining semantic fidelity and execution transparency. The results from all sub-agents are asynchronously aggregated and summarized by the main agent to produce the final answer. This route maximizes fine-grained schema interpretation and reasoning transparency, making it well-suited for tasks requiring detailed inspection of a limited number of elements.

3.3.2. Mode II – exploration-aggregation execution

When the task involves a large number of entities, direct asynchronous execution leads to high computational overhead and excessive token usage. To overcome this, the system activates the exploration-aggregation mode. In this mode, the main agent first samples a subset of entities for toolchain exploration. Their execution processes are monitored by an ObserverAgent, which extracts the reasoning pattern, i.e., the sequence of tools used for targeted field traversal. The discovered toolchain is then passed to a BulkExecutor, a Python-based executor that performs deterministic looped execution across all remaining entities without invoking the LLM, producing structured outputs. Subsequently, an AggregationAgent receives these outputs and invokes the LLM to generate Python code for final operations such as numerical aggregation, filtering, counting, and classification. This design significantly reduces token consumption and execution time by extracting a generalizable toolchain from a small sample and applying it via deterministic Python execution, thereby avoiding repeated LLM invocations for each entity. Notably, the mode assumes schema homogeneity, meaning entities of the same IFC type share identical field hierarchies.

In summary, the proposed IFC-Agent framework integrates multiple specialized components, including the main agent for planning and coordination, a set of sub-agents for schema-guided traversal, an ObserverAgent for extracting reusable toolchains, a BulkExecutor for Python-based non-LLM batch execution, and an AggregationAgent for LLM-based code generation and result aggregation. Through adaptive routing across these components, the system establishes a coordinated workflow that spans task decomposition, individual execution or exploration, observation, generalization, and aggregation. This dual-mode design enables the IFC-Agent to flexibly switch between fine-grained per-entity reasoning and large-scale batch processing while maintaining interpretability, scalability, and semantic consistency across diverse IFC query scenarios.

3.4. Task planning and adaptive execution

This section describes the task planning mechanism and the adaptive execution strategy of the IFC-Agent. Section 3.2.1 introduces the prompt-guided task understanding and decomposition process, while Section 3.2.2 presents the adaptive dual-mode execution mechanism for handling queries at different entity scales.

3.4.1. Task understanding and decomposition

The proposed IFC-Agent adopts a prompt-guided but lightweight approach for task understanding and decomposition, which integrates both rule-based preprocessing and prompt-guided reasoning. Because the system emphasizes execution adaptability instead of designing elaborate planning chains in the prior study [9], it directly translates a user query into structured subtasks that can be seamlessly handled by dual-mode executors. When a natural-language query is received, the system first employs the RewriteIfNeeded tool, a rule-based linguistic preprocessor designed to handle symbol-based or shorthand conditional statements that commonly appear in BIM-related queries. It utilizes regular-expression parsing to detect comparison operators (e.g., $>$, $<$, $\geq$, $=$) and rewrites them into natural-language descriptions. For example, a query such as *"Find all walls with depth $>$ 0.2 and width $<$ 0.5"* is rewritten as *"Find all walls whose depth is greater than 0.2 meters and whose width is smaller than 0.5 meters."* This step ensures syntactic and semantic consistency before task decomposition, as well as reducing the errors during LLM-based decomposition that can be triggered by symbolic notation, shorthand constraints, or ambiguous phrasing.

After normalization, the refined query is passed to the task-decomposition agent, which acts as a prompt-driven module for multi-hop task planning. The prompt follows a structured zero-shot paradigm augmented with illustrative examples, allowing the LLM to infer decomposition logic through rule-based instructions rather than direct

sample imitation. Given an input in the form of {"question": "<user query>"}, it generates a JSON-structured list of subtasks corresponding to the logical steps required for execution. For instance, a compound query such as *"What is the capability of the hot water boiler?"* is automatically decomposed into: {"subtask": "Find all hot water boilers", "subtask": "Find the capability of each hot water boiler"}. Then, each subtask is executed sequentially, where the output of the previous subtask provides the working scope for the next. This two-layer process of combining rule-based rewriting and prompt-guided decomposition preserves logical dependency and semantic consistency while enhancing the reliability of task decomposition.

3.4.2. Adaptive dual-mode execution

As introduced in Section 3.1, IFC-Agent executes queries via two complementary modes (or routes) determined by the entity-scale threshold. When the number of target entities is below the threshold, the system adopts Mode I – asynchronous fine-grained execution. In this setting, the main agent dispatches the subtasks to multiple asynchronous executors, each responsible for a specific domain such as geometry, quantity, property, or relation. These executors then launch sub-agents that perform schema-guided reasoning independently for each entity using the predefined tools. The agent reasoning process and multi-agent collaboration (i.e., main agent and sub-agents) follow the ReAct (Reasoning and Acting) framework [38], which is a "thought–action–observation" loop. Because the number of entities is small (less than the threshold), the reasoning traces, execution outputs, and intermediate steps generated by all sub-agents remain within the LLM context window. This enables the system to preserve complete interpretability and to aggregate all results directly under the main agent for final summarization and answer generation. Mode I is particularly effective for tasks that require flexible, entity-specific exploration. For example, when each entity involves different schema depths or varying field hierarchies, this design allows each sub-agent to autonomously traverse the schema and identify the most relevant fields to the user's query, thereby ensuring maximum flexibility and transparency. The pseudo code of Mode I is shown as below.

In contrast, when the number or scale of entities exceeds the threshold, the system activates Mode II – exploration–aggregation execution, which is optimized for large-scale or repetitive entity operations. Instead of processing every entity under LLM supervision, the system first selects a small subset of representative entities for schema-guided exploration. During this exploratory stage, asynchronous executors are still used to invoke the same predefined atomic tools as in Mode I, but their reasoning logs are subsequently analysed by the observer agent. The observer monitors the execution process and extracts a reusable toolchain pattern, i.e., the ordered sequence of tools and field access steps that achieved the correct reasoning outcome. To ensure the extracted toolchain is correct, the predefined tool names are injected into the observer prompt so that the agent can recognize and focus only on these valid tool invocations while ignoring executor-level actions. As a result, the observer summarizes a concise, schema-aligned tool chain that captures the essence of how the exploration tasks were completed. Once the toolchain is inferred, it is applied to the remaining entities through a deterministic bulk executor based on Python. This executor replays the same reasoning pattern across all entities without additional LLM calls, thereby eliminating repeated API requests and token consumption. The outputs from this process are formatted and truncated (such as head = 50, tail = 50), which preserves the entity IDs (i.e., GlobalIds) and their corresponding field values, as well as compress redundant intermediate text. These structured outputs are then delivered to the aggregation agent, which employs LLM-generated code to perform numerical operations, attribute filter, counting, classification, and summarization. This approach greatly enhances the reliability of answer generation compared to end-to-end code generation directly operating on raw IFC models. The pseudo code of Mode II is shown as below.

3.5. Tool design, invocation, and assignment

This section introduces the design of the tool framework in IFC-Agent and explains how tools are invoked and assigned across different agents. Section 3.3.1 describes the design principles and functional coverage of the developed tools, Section 3.3.2 explains the tool invocation patterns and execution chains during reasoning, and Section 3.3.3 presents the agent–tool assignment mechanism and safety constraints that ensure controlled and interpretable tool usage.

3.5.1. Tool design principles and functional coverage

The tools' development is grounded in the IFC schema and designed to align with human parsing of nested STEP representations while ensuring consistent tool invocation. For example, the tool pair *GetInfoByStepId* and *QueryWithStepIdDict* enables recursive traversal through fields such as *ObjectPlacement*, *Representation*, and *RelatedElements*, thereby supporting iterative decomposition of complex IFC structures. This schema-guided strategy promotes flexibility in multi-layer reasoning while avoiding over-specification across similar entity types. In contrast, as for fields like *IfcPropertySet* and *IfcElementQuantity*, although they have a tree-like structure, they follow relatively stable and shallow access patterns within the IFC schema. Therefore, these structures are handled by deterministic, hard-coded tools like *ExtractPropertySetByGlobalId* and *ExtractQuantitySetByGlobalId*, to enhance operational efficiency and ensure consistent alignment with *IfcOpenShell*'s domain-specific APIs.

All tools' definitions conform to LangChain's Tool interface, which consists of three components: *name*, *func*, and *description*. This structure supports precise invocation, functional encapsulation, and reasoning transparency. Notably, the *description* field is parsed by the LLM agent within the ReAct reasoning loop to infer each tool's purpose, input/output specification, and usage context. Therefore, a structured and detailed description serves not only as internal documentation, but also as an instruction template affecting tool selection. An illustrative example is provided in Fig. 2.

The developed tools are organized into five functional categories based on their roles in semantic reasoning and model-level operations: (i) information extraction tools, which recursively parse IFC entities and resolve nested STEP expressions; (ii) property and quantity tools, which retrieve values from *IfcPropertySet* and *IfcElementQuantity*; (iii) relation navigation tools, which traverse semantic links such as *Relating* and *Related* to support structural, spatial, and classification queries; (iv) modification and export tools, which edit IFC fields and export updated models; (v) control and support tools, which handle memory/cache querying, task rewriting, and result logging. Table 1 summarizes these functional categories along with representative tools and their typical usages. All the tools in the system follow a unified input-execution-

```
get_info_by_globalid = Tool(
    name="GetInfoByGlobalId",
    func=get_info_by_single_globalid,
    description=(
        "Perform a one-time get_info() on the IFC element with the given GlobalId.\n\n"
        "Extracts and stores basic IFC object fields into globalid_dict[<GlobalId>]
['BasicInfo'], including:\n"
        "- type, GlobalId, Name, Description, Tag, PredefinedType\n"
        "- ObjectPlacement, Representation, HasAssociations, ContainedInStructure\n\n"
        "Input must be a Python dict or JSON object with the following keys:\n"
        "- 'global_id': string (single GlobalId to query)\n\n"
        "Output: Success message. Fields are written to globalid_dict under 'BasicInfo'."
    )
)
```

Fig. 2. Example of tool definition in IFC-Agent.

Table 1
Functional categories and representative tools in IFC-Agent.

Category	Purpose	Example Tools	Typical Usage
Entity Info	Basic entity parsing	*GetInfoByGlobalId, GetInfoByStepId*	Extract core IFC fields, e.g., *Name, Tag, Description, ObjectPlacement.*
Property Set	Property extraction	*ExtractPropertySetByGlobalId*	Retrieve specific properties such as *FireRating, LoadBearing.*
Quantity Set	Quantity extraction	*ExtractQuantitySetByGlobalId*	Retrieve quantitative attributes such as *Volume, GrossArea.*
Relation	Relation traversal	*ExtractRelationsByGlobalId, DescribeRelationStep, DecideRelationNextHop*	Navigate relational links, e.g., *ContainedInStructure, HasAssociations.*
Modification	Field editing	*SetFieldByStepId, ExportModifiedIFC*	Modify entity fields and export IFC models.
Control	Task flow	*QueryMemory, RewriteIfNeeded QueryWithGlobalIdDict QueryWithStepIdDict*	Recall previous results, retrieve cached IFC data, and perform question rewriting when necessary.

output paradigm, which ensures consistent invocation and deterministic behaviour across agents. Each tool accepts well-structured inputs (in Python dictionaries or JSON format) and returns formatted outputs as structured observations within the ReAct reasoning loop. This unified design provides a transparent and extensible foundation for interpretable execution and modular augmentation.

3.5.2. Invocation patterns and execution chain

Tool invocation in IFC-Agent relies on three complementary patterns:

1) **Hard-coded tools** are deterministic tools based on IfcOpenShell, used for schema-resolved operations such as recursive field parsing, relation traversal, and model modification. They provide reproducible behaviour and low variance across sessions and are the default choice when the target field/route is structurally well defined (e.g., *ObjectPlacement*, *Representation*, *RelatedElements*).
2) **Prompt-guided soft tools** are applied when the base LLM cannot reliably infer the correct traversal path within the IFC schema, or when multiple plausible schema routes exist for the same target field. These prompts encode IFC schema-level constraints and human-defined preferences derived from modelling conventions, rather than relying on exact keywords or query-specific heuristics. These preferences specify which schema paths should be prioritised, discouraged, or treated as fallbacks, corresponding to how human experts reason about IFC semantics. For example, when extracting the field *Depth*, the prompt can instruct the agent to prioritise the geometric definition (*IfcShapeRepresentation* → *IfcExtrudedAreaSolid* → *Depth*) over the approximate bounding dimension (*IfcShapeRepresentation* → *IfcBoundingBox* → *ZDim*). As a result, semantically similar user queries are handled consistently, since traversal decisions are governed by schema-aware constraints rather than lexical matching, ensuring that the reasoning aligns with the physical structure of the model.
3) **LLM-generated code execution** is applied selectively for well-scoped computational subtasks that cannot be resolved through existing tool invocations alone and require post-processing over structured intermediate results. In such cases, code generation is restricted to basic arithmetic and deterministic data manipulation, such as addition, multiplication, aggregation (e.g., sum, min/max), ordering, or matrix composition over explicitly provided numerical inputs. The generated code is produced by prompting the LLM with structured intermediate results and fixed computation objectives, which leads to concise Python snippets that implement predefined operations rather than free-form logic. Typical examples include multiplying transformation matrices derived from chained *IfcLocalPlacement* hierarchies or aggregating numerical attributes extracted from IFC entities. This constrained design limits the scope of code generation and improves execution reliability and reproducibility.

Unlike earlier LLM-assisted BIM systems [4,10], where toolchains were rigidly hard-coded into predefined procedural templates, the proposed IFC-Agent constructs reasoning chains dynamically through schema-guided traversal. Each tool is modular and schema-aligned, allowing the agent to flexibly compose and reorder tools according to the query context. This design supports adaptive sequencing, i.e., the agent determines which tool to invoke next by analysing field hierarchies, reference depths, or returned STEP structures, thereby ensuring interpretability throughout multi-hop reasoning. Moreover, the toolset remains extensible through augmentation, allowing newly added tools to be seamlessly integrated for handling unforeseen tasks, thereby maintaining scalability as the system evolves.

During execution, tools are dynamically composed into chains within the ReAct loop, where each tool's output serves as structured observations and provides contextual input for the next reasoning step. For instance, when retrieving spatial placement information, the agent first calls *GetInfoByGlobalId* to access the *ObjectPlacement* field, then resolves the referenced entity via *GetInfoByStepId*, and finally extracts coordinates using *QueryWithStepIdDict*. This stepwise chain-of-tools reasoning emulates how human experts parse IFC structures, i.e., reasoning through relational dependencies rather than performing predefined lookups, and enables consistent multi-hop semantic traversal. Moreover, many IFC relationships, such as spatial containment, openings, or classification references, are not directly embedded within an element but are represented through intermediate relational entities, e.g., *IfcRelContainedInSpatialStructure*, *IfcRelVoidsElement*, *IfcRelAssociatesClassification*. In such cases, the agent continues chaining tools in a schema-guided order, iteratively resolving the *Relating* and *Related* fields of these entities to reach the final target. This nested traversal mechanism enables IFC-Agent to uncover implicit dependencies with full semantic transparency, achieving more reliable and consistent results than ad-hoc prompting or isolated retrieval-based approaches.

3.5.3. Agent-tool assignment and safety constraints

To ensure predictable, interpretable, and domain-specific behaviour, IFC-Agent adopts a static agent–tool binding mechanism. Each agent is assigned a fixed and domain-aligned toolset during initialization, which constrains its operation to a well-defined functional scope. This bounded design acts as a built-in safety layer that prevents unintended tool usage, cross-domain interference, or over-privileged modification commands. All toolsets are statically registered during agent initialization, and each tool's description serves as a semantic reference for LLM planning and validation. This ensures that the LLM can only invoke tools explicitly allowed within the current agent's domain. For example, the geometry agent is restricted to placement resolution and STEP traversal tools, whereas the property agent only operates on the tools for *IfcPropertySet* extraction. This mechanism maintains precision and interpretability in tool invocation, ensuring controlled and semantically consistent reasoning behaviour. The examples of modular agents and their assigned tools are listed in Table 2.

3.6. Dual memory for IFC reasoning

To support both procedural continuity and semantic reusability during IFC reasoning, IFC-Agent adopts a dual-memory architecture composing task-level procedural memory and entity-level semantic caches. The procedural memory (i.e., *CustomMemory*) preserves the

Table 2
Examples for modular agents and assigned tools.

Agent Type	Task Domain	Assigned Tools
Main Agent	Task planning & coordination (query decomposition, memory querying, executor dispatch, and result management)	*LoadIFCFile, QueryMemory, TaskDecompositionAgent, ObserverAgent, BulkExecutor, RewriteIfNeeded, SaveFinalAnswer, ExportModifiedIFC, PythonExecutor*
BasicInfo Agent	Basic IFC entity information extraction	*GetInfoByGlobalId, QuerySingleGlobalIdDict, GetInfoByStepId, QueryWithStepIdDict, SetFieldByStepId*
Geometry Agent	Spatial placement and geometric reasoning	*GetInfoByStepId, QueryWithStepIdDict, ComputePlacementMatrixFromStepId, SetFieldByStepId*
Quantity Agent	Quantity extraction (area, volume, and dimensional data)	*ExtractQuantitySetByGlobalId, QuerySingleGlobalIdDict, SetFieldByStepId*
Property Agent	Property set reasoning and associated relation traversal	*ExtractPropertySetByGlobalId, ExtractRelationsByGlobalId, GetInfoByStepId, QueryWithStepIdDict, SetFieldByStepId*
Relation Agent	Semantic and connectivity relations (e.g., openings, assignments, connectivity)	*ExtractRelationsByGlobalId, ExtractOpeningsByGlobalId, ExtractConnectivityRelationsByGlobalId, ExtractAssignmentRelationsByGlobalId, DescribeRelationStep, DecideRelationNextHop*
SpaceBoundary Agent	Room boundaries and adjacency detection	*ExtractBoundariesByGlobalId, FindAdjacentSpacesByElement, DescribeRelationStep, DecideRelationNextHop, GetInfoByStepId, QueryWithStepIdDict*
SpatialContainment Agent	Storey-level spatial containment and hierarchy resolution	*ExtractContainsElementsByGlobalId, GetInfoByStepId, QueryWithStepIdDict, SetFieldByStepId*

agent's reasoning traces and tool-level summaries, enabling reliable recall of prior actions and outcomes. The semantic cache, built on global dictionaries (i.e., *globalid_dict* and *stepid_dict*), maintains structured representations of parsed IFC entities for consistent and cross-agent reuse. Unlike previous frameworks such as Text2BIM [6,7], which primarily capture session-level task history, the proposed dual-memory design extends memory to the entity level, allowing previously parsed IFC objects to be directly reused across reasoning steps. This fine-grained approach improves semantic consistency, reduces redundant parsing, and preserves interpretable parsed entity information across tasks and agents. Prior work [11] has demonstrated that this dual-memory architecture can reduce execution time by over 60% through cross-session reuse of procedural context and parsed entities.

3.6.1. Procedural memory for task context

The procedural memory module (i.e., *CustomMemory*) extends the standard conversational buffer used in LangChain by introducing an additional *procedure_context* for logging tool-level execution traces, as shown in Fig. 3. Unlike conventional memory module only retains dialogue histories, this customized module records detailed metadata for each tool invocation, including the tool name, execution timestamp, and output summary, thereby keeping a record of the agent's reasoning procedure, as illustrated in Fig. 4. At the beginning of each new query,

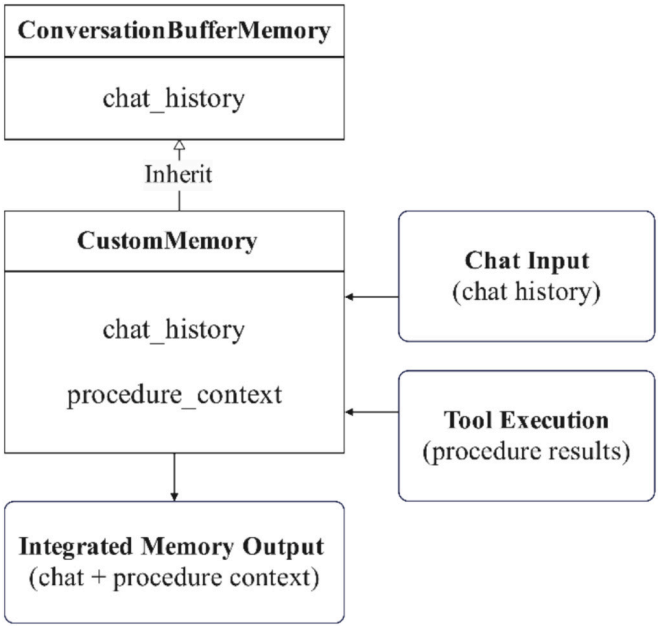


Fig. 3. *CustomMemory* module and data flow.

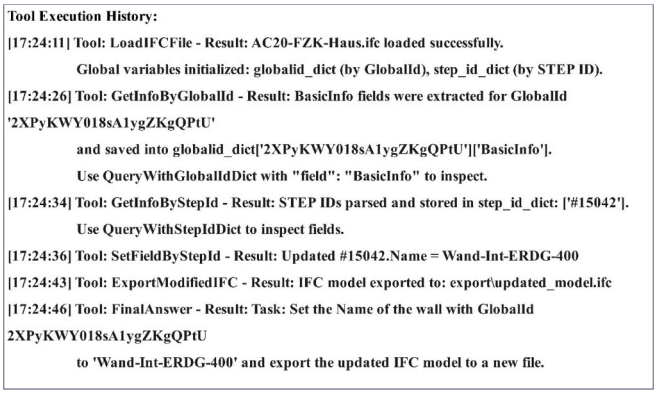


Fig. 4. Example of tool execution history recorded in the *CustomMemory* module.

the main agent invokes the *QueryMemory* tool to check whether relevant task results already exist. If matched, the outputs are directly reused, thereby reducing redundant computation. In this way, the procedural memory supports cross-session persistence of task knowledge and provides an auditable trail of the system's decision process.

3.6.2. Semantic cache for entity access

To enable efficient reuse of parsed IFC entities during reasoning, IFC-Agent maintains shared semantic caches composed of two synchronized global dictionaries, i.e., *globalid_dict* and *stepid_dict*. The *globalid_dict* stores high-level entity data indexed by GlobalIds, whereas the *stepid_dict* retains fine-grained intermediate structure referenced by STEP IDs. During reasoning, an agent first queries these caches through tools such as *QueryWithGlobalIdDict* or *QueryWithStepIdDict*. If unavailable, it triggers *GetInfoByGlobalId* or *GetInfoByStepId* to retrieve the required information and update the caches. This query-parse-reuse loop, driven by prompt instructions (shown in Fig. 5), minimizes redundant parsing and ensures consistent access to IFC entities across sub-agents. For example, when different agents access the same element, such as an *IfcWall*, they obtain identical structured representation through the shared caches, thereby enhancing both efficiency and consistency. Furthermore, unlike approaches such as IFC-Graph [2], which require

Example 1 - GlobalIdDict

1. Always begin by checking if 'BasicInfo' exists in globalid_dict:
   Action: QueryWithGlobalIdDict
   Action Input: {"global_id": "<id>", "field": "BasicInfo"}
2. If not found, run:
   Action: GetInfoByGlobalId
   Action Input: {"global_id": "<id>"}
3. Then repeat step 1 to extract the required field from BasicInfo.

Example 2 - StepIdDict

1. Always begin by checking if the field exists in stepid_dict:
   Action: QueryWithStepIdDict
   Action Input: {"step_ids": ["#xxxx"], "field": "Depth"}
2. If not found, run:
   Action: GetInfoByStepId
   Action Input: {"step_ids": ["#xxxx"]}
3. Then repeat step 1 to extract the field.

Fig. 5. Prompt templates for memory access using GlobalId and STEP ID.

heavy model preprocessing, this incremental cache design allows the system to dynamically construct an IFC graph during reasoning, where parsed entities act as nodes, semantic relations form edges, and extracted field values become attributes. This on-demand strategy reduces preprocessing overhead and aligns naturally with the stepwise reasoning workflow.

4. Experimental validation

This section presents the experimental validation of the proposed IFC-Agent framework. Section 4.1 introduces the experimental setup, including system configuration, benchmark models, and query design. Section 4.2 reports the scalability analysis used to determine the threshold for adaptive mode switching. Section 4.3 presents the overall benchmark performance across different LLM models, and Section 4.4 analyses the benchmark results in detail.

4.1. Experiment setup

The following experiments were conducted under a unified configuration to ensure reproducibility. Section 4.1.1 details the LLM configuration and system execution settings, and Section 4.1.2 presents the benchmark IFC models and query design.

4.1.1. LLM configuration and system execution setup

The IFC-Agent is implemented in Python using the LangChain framework and follows the ReAct paradigm for tool-augmented reasoning. In the proof-of-concept experiments, the system is instantiated with GPT-series large language models accessed via the OpenAI API. All experiments use a temperature parameter of 0 to ensure deterministic and reproducible behaviour, while other generation parameters follow OpenAI API defaults, including top-p equal to 1.0, presence penalty and frequency penalty both set to 0.0, model-dependent maximum token limits, and no explicit stop sequences. The agent execution follows the ReAct reasoning loop with a maximum of 25 reasoning steps per task.

The system does not rely on OpenAI's structured output mode. Instead, tool invocation is implemented through structured prompting and a customized JSON parsing mechanism [11]. This design allows the LLM to flexibly interleave natural language reasoning with structured function calls while maintaining robust execution. The system also employs an adaptive dual-mode execution strategy based on the number of target entities, where fine-grained entity-level execution is applied for small-scale tasks and exploration-aggregation execution is activated as the entity count increases. This threshold is determined empirically through scalability analysis detailed in Section 4.2.

4.1.2. Benchmark models and query design

Three representative IFC models, namely AC20-FZK-Haus.ifc [39], Schependomlaan.ifc [40], and Duplex-MEP.ifc [41], are selected to validate the proposed IFC-Agent, as illustrated in Fig. 6. These models are chosen to cover a diverse range of building typologies and semantic characteristics, including architectural components, spatial structures, construction sequencing, and mechanical, electrical, and plumbing (MEP) systems.

Rather than directly reusing existing benchmark question sets, the IFC query benchmark in this paper is designed from a system-oriented perspective to reflect IFC-native execution complexity. The design is informed by prior IFC query taxonomies, such as the 11 task categories used in GQL4BIM [3], but the queries are reorganized along three complementary dimensions that more directly capture the challenges faced by tool-augmented, schema-aware reasoning systems, i.e., schema traversal depth, constraint and execution complexity, and execution aim. Schema traversal depth indicates the number of IFC relational hops required for query resolution, ranging from 0-hop (direct attribute access) to 3+ hops (deep recursive traversal). Constraint and execution complexity captures the degree of reference navigation, conditional filtering, multi-step dependency, and execution intensity involved in the query, spanning from simple retrieval to execution-intensive tasks. Execution aim describes the primary query-level operations performed, such as retrieval, traversal, extraction, tracing, filtering, comparison, aggregation, or modification.

Table 3 summarizes the IFC query benchmark design categories and illustrates how the queries range from simple attribute-level access to

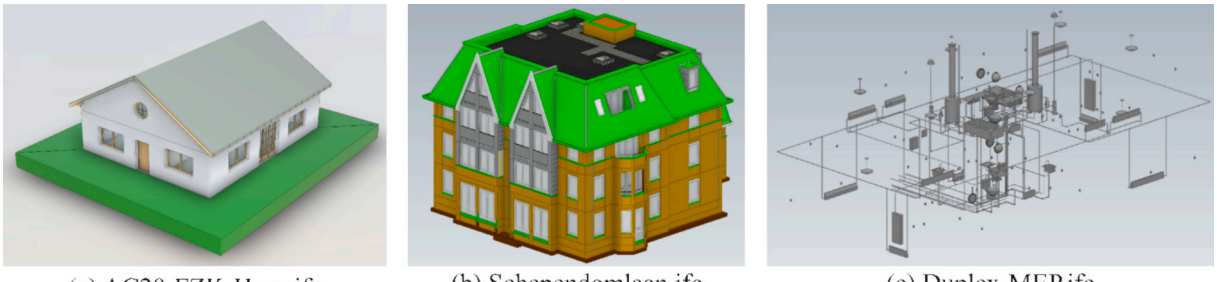


Fig. 6. Selected IFC models for validating the IFC-Agent system.

Table 3
IFC query benchmark design categories.

Dimension	Categories	Description (Concise)
Traversal Depth	0 hop	Direct access to entity attributes without following any schema-defined IFC references.
	1–2 hop	Short traversal across one or two schema-defined IFC references.
	2–3 hop	Chained traversal across multiple schema-defined IFC references.
	3+ hop	Deep or recursive traversal of IFC structures via schema-defined references.
Constraint & Execution Complexity	Simple Retrieval	Direct attribute access with no reference traversal.
	Relational Navigation	Explicit reference traversal with limited constraints.
	Multi-hop Reasoning	Multi-step reasoning across multiple IFC references.
	Constraint Filtering	Multiple or compound logical constraints.
	Execution-intensive	Queries involving long execution chains, deep reference traversal, or complex computation.
Execution Aims (sequence)	Retrieval	Retrieve entities or attribute values.
	Traversal	Navigate schema-defined IFC references.
	Extraction	Extract structured information from referenced entities.
	Tracing	Follow reference paths to reconstruct derivation chains.
	Filtering / Comparison	Apply constraints or comparisons during execution.
	Aggregation	Perform grouping or aggregation operations.
	Modification	Modify IFC entities and produce updated outputs.

execution-intensive tasks involving deep, multi-hop reasoning over IFC structures. In this benchmark, a hop is defined strictly as a single traversal along a schema-defined IFC reference, such as between entities or their associated attribute objects, rather than an API call or an execution step. Queries that access only direct entity attributes without following any schema-defined references are categorized as 0-hop, while each additional traversal along IFC entity or attribute references incrementally increases the hop depth. Constraint and execution complexity is organized from simple retrieval, to relational navigation and multi-hop reasoning with increasing reference dependencies, then to constraint filtering with compound logical conditions, and finally to execution-intensive queries, which require extended execution chains. Although these categories correspond to different sources of complexity, they broadly reflect progressively greater execution demands across the benchmark. Unlike traversal depth or execution complexity, execution aims do not represent increasing levels of difficulty. Instead, a query can involve multiple aims executed in sequence (e.g., retrieval → traversal → filtering → modification).

The complete list of benchmark queries is provided in Appendix A. Queries 1–52 are executed on the architectural model AC20-FZK-Haus.ifc, which primarily covers building elements, spatial structures, material assignments, and geometric representations. Queries 53–58 are evaluated using Duplex-MEP.ifc model, focusing on mechanical, electrical, and plumbing (MEP) components such as ducts, pipes, boilers, and flow terminal devices. The remaining queries, namely Queries 59 and 60, are conducted on the Schependomlaan.ifc model, which includes construction sequencing and task-related information.

4.2. Scalability analysis and threshold calibration

This scalability analysis aims to investigate the performance boundaries of the fine-grained entity-level execution strategy (Mode I) and determine the optimal threshold for adaptive mode switching. To isolate the effect of entity scale on execution reliability, the experiments are conducted under a fixed execution route where Mode I is applied throughout the entire workflow without adaptive switching. The IFC-Agent is instantiated using GPT-4o under the same system configuration described in Section 4.1.1. A set of 9 representative queries is designed to cover six major IFC component categories, with task complexities spanning from simple attribute retrieval to per-entity computation, grouping, and aggregation. These queries are evaluated on two IFC models, namely Duplex-ARC and Duplex-MEP. For each query, the number of target entities is obtained using the *GetGlobalIdsByAttributeValue* tool, which returns both the entity count and the complete list of GlobalIds for the specified IFC entity type (e.g., *IfcWall*, *IfcWindow*). The complete list of queries and their execution outcomes are reported in Appendix B.

The results reveal a clear degradation pattern of Mode I as the number of target entities increases. When the task involves a small number of entities (fewer than 20), Mode I reliably completes the full execution workflow, including per-entity extraction, grouping, and simple aggregation. For example, queries involving 2 staircases, 8 beams, and 14 doors all achieved successful execution under Mode I. At a moderate scale (approximately 20–25 entities), Mode I begins to exhibit instability. Query 5, involving 24 windows, succeeded in grouping and counting, but Query 6, which requires per-entity computation and sum aggregation over the same 24 windows, produced incorrect results due to misaligned entity-value associations. As the number of entities further increases beyond 50, Mode I consistently fails to complete tasks as specified. Query 7 with 56 walls, Query 8 with 105 flow terminals, and Query 9 with 427 flow segments all resulted in execution failures, where the LLM struggled to maintain long-horizon, fine-grained execution over large entity sets. These observations indicate that fine-grained entity-level execution (Mode I) does not scale well with increasing data volume and execution complexity. In contrast, all queries that produced incorrect or failed results under Mode I were completed successfully through the exploration-aggregation workflow in Mode II, as shown in the recovery column of Appendix B. This demonstrates Mode II's improved robustness and scalability for multi-entity and execution-intensive IFC queries. Moreover, Mode II significantly reduces token consumption and the number of LLM invocations when handling large entity sets, thereby enhancing overall execution efficiency. Based on these findings, a threshold of 20 entities is adopted for adaptive mode switching in the full benchmark evaluation to balance execution reliability and reasoning transparency.

4.3. Overall benchmark performance

The benchmark experiments were conducted using three OpenAI LLM models with varying reasoning capabilities: GPT-3.5-turbo, GPT-4o-mini, and GPT-4o. All models were evaluated under identical experimental settings, including the same IFC-Agent toolsets, prompting strategies, and adaptive dual-mode execution with a mode switching threshold of 20 entities.

4.3.1. Evaluation metrics and success criteria

The ground truth for benchmark query answers was established manually using the IfcOpenShell library and Python scripts, and the results were cross validated via usBIM IFC viewer [42]. To systematically assess the IFC-Agent's performance across the query benchmark, task completion success rate is adopted as the primary evaluation metric, defined as:

$$Success\ Rate = \left(\frac{N_{successful}}{N_{total}}\right) \times 100\% (1)$$

where $N_{successful}$ is the number of queries that produce correct results, and N_{total} is the total number of benchmark queries.

A query is considered successful if it satisfies both of the following

conditions:

1) The agent reaches a final answer without timeout (maximum 25 reasoning steps) or execution errors, including tool invocation failures and invalid schema navigation.
2) The answer matches the ground truth, where the verification criteria vary by query type. For retrieval queries, success requires exact match of the returned entity set with no false positives or negatives; for attribute extraction queries, all target entities must be processed and extracted values must match IFC field values; for numerical aggregation queries, both the correct numerical value and correct entity identification are required; for multi-hop reasoning queries, the complete relationship traversal chain must be executed, and correct final entities must be retrieved; for grouping queries, all groups must be identified with correct entity assignment per group. For modification queries, the exported IFC file must pass schema validation and target fields must be correctly updated.

4.3.2. Benchmark performance analysis

Benchmark experiments were conducted using three OpenAI LLM models: GPT-3.5-turbo, GPT-4o-mini, and GPT-4o. All models were evaluated under identical experimental settings, including the same IFC-Agent toolsets, prompting strategies, and adaptive dual-mode execution with a mode switching threshold of 20 entities. The detailed execution results for all benchmark queries are provided in Appendix A. Fig. 7 presents a consolidated view of benchmark results across the three models, where Fig. 7(a) summarizes the overall success rate, Fig. 7(b) illustrates success rates stratified by schema traversal depth, and Fig. 7 (c) shows success rates stratified by constraint and execution complexity. Performance is not analysed separately by execution aims because success rates are evaluated at the query level rather than at intermediate execution steps. Since most queries involve multiple aims executed in sequence, analysing individual aims would require step-level inspection of intermediate reasoning and tool interactions, which is beyond the scope of this paper.

As can be seen from Fig. 7, GPT-4o successfully completed all benchmark queries across varying levels of complexity, including tasks involving multi-hop traversal and reasoning, as well as diverse execution aims. This demonstrates that the proposed IFC-Agent framework, when instantiated with sufficiently capable LLMs, can reliably handle execution-intensive tasks requiring deep schema navigation, constraint satisfaction, and large-scale entity processing. In contrast, GPT-3.5-turbo succeeds only on simple retrieval tasks with 0-hop traversal and fails consistently once schema traversal or multi-step execution is required. This limitation reveals a fundamental inadequacy in task planning capabilities. Specifically, GPT-3.5-turbo often generates incomplete or incorrect execution plans that fail to account for the multi-step nature of schema-constrained reasoning. Moreover, it relies on one-shot invocations of hard-coded tools, treating each tool call as an isolated operation rather than part of a coherent reasoning chain. This prevents further schema traversal or resolution of referenced fields even when explicit prompts are provided, as the model lacks the ability to maintain execution context across multiple tool invocations. As a result, it is unable to progress beyond basic instance-level lookups or perform even single-hop relational traversal, indicating that effective IFC querying requires not only tool access but also the reasoning capacity to orchestrate tools in schema-aware sequences.

GPT-4o-mini demonstrates substantially stronger performance than GPT-3.5-turbo and can handle a wide range of relational and multi-step IFC queries. However, as shown in Fig. 7(b) and (c), its performance degrades noticeably with increasing schema traversal depth and execution complexity. This degradation is primarily attributed to weaker task planning and execution robustness compared with GPT-4o. In practice, GPT-4o-mini is more prone to stalling during complex traversal and producing incomplete or misaligned results in aggregation or calculation tasks, suggesting difficulties in maintaining accurate entity-value mappings across extended execution sequences. Although GPT-4o-mini can invoke auxiliary executors such as the ObserverAgent and BulkExecutor, it often fails to construct correct toolchains or generate reliable executable code, especially when handling constraint-heavy or execution-intensive queries that require precise coordination among multiple execution steps. These observations indicate that while the proposed agentic framework can improve LLM performance through tool augmentation, there remains a performance ceiling determined by the model's underlying reasoning capacity.

5. Case studies

This section presents four representative case studies that validate the IFC-Agent's capabilities across different task types. Section 5.1 demonstrates field-level querying and modification, Section 5.2 illustrates task decomposition and exploration-aggregation execution, Section 5.3 validates coordinate transformation through placement hierarchy reasoning, and Section 5.4 shows incremental IFC graph construction driven by query execution.

5.1. Field-level query and modification

The IFC-Agent supports fine-grained querying and modification of entity fields through schema-guided navigation, which serves as the foundation for complex reasoning tasks. This capability enables precise access to deeply nested IFC data and direct model editing while maintaining schema compliance.

5.1.1. Schema-guided field query and prompt constraint

Depending on data complexity, the system employs two complementary approaches. For top-level attributes such as *Name*, *Description*, or *PredefinedType*, as well as quantities and properties accessible through *QuantitySets* and *PropertySets*, predefined tools such as *GetInfoByGlobalId*, *ExtractQuantitySetByGlobalId*, and *ExtractPropertySetByGlobalId* provide direct access with straightforward execution. For more complex, nested data relationships, the agent performs schema-guided semantic tracing using a sequence of atomic tools, including *GetInfoByStepId*, *QueryWithStepIdDict*, *DescribeRelationStep*, and *DecideRelationNextHop*, to navigate the IFC hierarchy step by step and retrieve information not directly accessible from the queried entity. This approach leverages the implicit schema awareness of advanced LLMs such as GPT-4o, and the reasoning performance can be further enhanced through prompt constraints that guide the agent to select appropriate traversal paths.

In practice, multiple valid traversal paths often exist for retrieving the same type of information. For example, extracting the extrusion depth of a wall may involve either of the following paths:

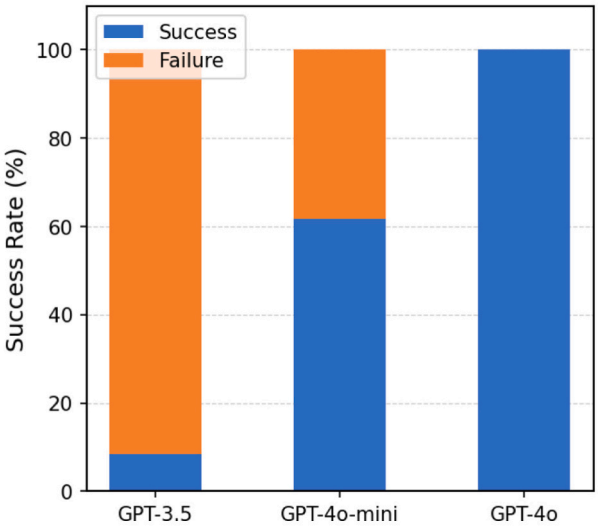
- *IfcShapeRepresentation → IfcExtrudedAreaSolid → Depth*
- *IfcBooleanClippingResult → FirstOperand → IfcExtrudedAreaSolid → Depth.*

Additionally, LLMs can heuristically infer values like *ZDim* of *IfcBoundingBox* as the extrusion depth based on element type (e.g., *IfcWall*):

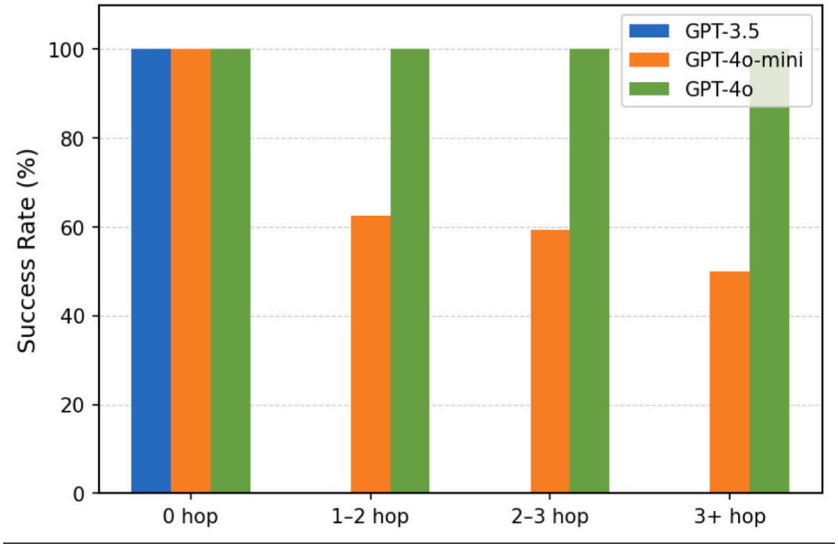
- *IfcRepresentation → IfcProductDefinitionShape → IfcShapeRepresentation → IfcBoundingBox → XDim / YDim / ZDim.*

Another example is the variation in wall thickness based on the representation approach:

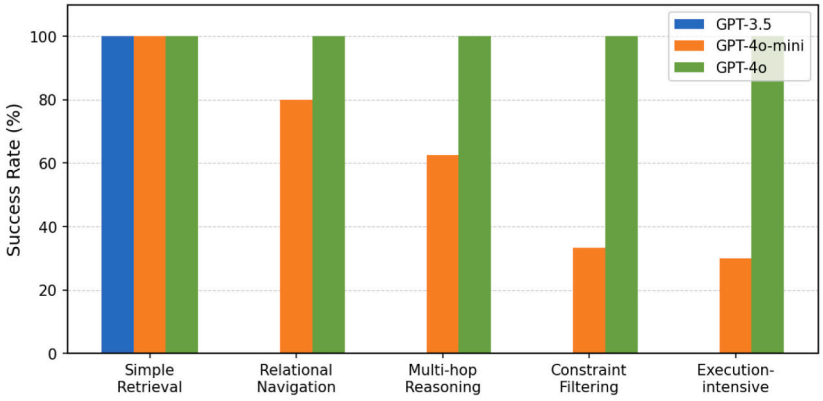
- Geometry-based: *IfcExtrudedAreaSolid → SweptArea → IfcRectangleProfileDef → YDim*
- Material-based: *IfcMaterialLayerSetUsage → ForLayerSet → MaterialLayers[n] → LayerThickness.*



(a) Overall success rate by different LLMs



(b) Success rate by traversal depth



(c) Success rate by constraint & execution complexity

Fig. 7. Benchmark results of IFC query execution across different LLMs.

Therefore, when multiple paths exist, prompt constraints are necessary to ensure alignment of traversal paths with representation conventions, project specifications, or stakeholder preferences.

5.1.2. Executor selection and field modification

The system routes queries to specialized sub-agents based on task category. Geometry-related queries are handled by the *SequentialGeometryExecutor*, while material or property-related queries are processed by the *SequentialRelationExecutor* or *SequentialPropertyExecutor*. This selection uses a rule-based prompt where keywords from the task description (e.g., height, material, placement) are mapped to predefined categories, as illustrated in Fig. 8. The design is modular and extensible, allowing new executors to support additional BIM domains such as HVAC analysis, structural reasoning, or cost estimation.

Field modification follows the same schema-guided navigation process. Once the target field is identified, the agent modifies its value and exports the updated IFC model. For example, given the query "Find the material of wall '3PfS_Y_DBAfq5naM6zD2Z' and change the material layer thickness to 0.3m. Then, export the updated IFC file," the system navigates the following schema paths:

- To identify the material name: *IfcRelAssociatesMaterial → IfcMaterial → Name*.
- To locate the thickness field: *IfcRelAssociatesMaterial → IfcMaterialLayerSetUsage → IfcMaterialLayer → LayerThickness*.

After identifying the target field's STEP ID, the agent applies the modification and exports an updated IFC model that passes schema validation. A detailed execution trace of this process is provided in Appendix C. This capability enables practical BIM editing workflows and distinguishes the IFC-Agent from query-only approaches.

5.2. Task decomposition and exploration-aggregation execution

This section illustrates how the IFC-Agent handles complex queries through task decomposition and exploration-aggregation execution. Section 5.2.1 describes the decomposition process, and Section 5.2.2 demonstrates the exploration-aggregation workflow through a representative example.

5.2.1. Task decomposition

Task decomposition, executed through the *TaskDecompositionAgent*, breaks down a natural-language query into a sequence of ordered subtasks. Each subtask's output serves as input for the next, ensuring a continuous execution chain for the full query. The *TaskDecompositionAgent* uses general prompt principles and zero-shot exemplars to perform this process. These principles help the agent infer decomposition logic based on high-level reasoning rules instead of relying on predefined templates, such as analysing query complexity, ensuring clarity and logical order, and maintaining logical dependencies across subtasks, as illustrated in Fig. 9.

A small set of zero-shot exemplars is used to guide the agent in understanding the structural pattern of task decomposition without relying on explicit templates. These generic examples (e.g., "Find all walls" → "Find the area of each wall" → "Calculate the total area of walls") illustrate the reasoning logic expected. Notably, these examples are not tied to any specific query and must not include the target query to be decomposed. This ensures that the agent infers how to decompose unseen queries based on underlying logic, rather than pattern matching against predefined templates.

For instance, the query "What's the maximum diameter of the drainage pipes?" would be decomposed as follows:

- *Subtask 1: Find all drainage pipes.*
- *Subtask 2: Find the diameter of each drainage pipe.*
- *Subtask 3: Determine the maximum diameter among the drainage pipes.*

5.2.2. Exploration-aggregation execution

For tasks or subtasks involving multiple target entities exceeding a given threshold (e.g., 20), the system activates Mode II, a standardized Exploration-Aggregation workflow. Under the assumption of a homogeneous schema structure, this workflow enables consistent reasoning across large-scale entity sets by exploring a small sample of entities (typically 1–2), then generalizing the reasoning patterns (i.e., toolchains or field chains) to all remaining entities.

- For example, for Subtask 1: Find all drainage pipes.

Given that there are 427 *IfcFlowSegment* entities (over the threshold), the system first performs autonomous exploration on 1–2 sampled entities. During this stage, the ObserverAgent analyses the execution log and summarizes the following toolchain pattern:

{"step": 1, "tool": "GetInfoByGlobalId", "input_schema": {"global_id": "<gid>"}},

{"step": 2, "tool": "QueryWithGlobalIdDict", "input_schema": {"global_id": "<gid>", "field": "BasicInfo.Name"}.

The BulkExecutor then applies this schema-aligned toolchain to all remaining entities, generating a structured output. A few representative lines are listed below:

"1xtfyUl7TDzuHsgAvJTR2z": "- BasicInfo.Name:\n Pipe Types:Mechanical Pipe:557586",

"3eAA$CJlT1EeOPz$$sQNW2": "- BasicInfo.Name:\n Pipe Types: Waste:582953",

"1jfuKVLvDAvujmZd$g5KIv": "- BasicInfo.Name:\n Pipe Types: PVC:605354",

"1VltIYtz91U9QZX8sofKE$": "- BasicInfo.Name:\n Pipe Types:Hot Water:594105".

Next, these results are passed to the AggregationAgent, which generates filtering code automatically based on the task description. Following the prompt instruction *"Drainage Pipe → Name contains ['waste', 'soil', 'sewer', 'sanitary']"*, the LLM (i.e., GPT-4o) produces the following executable code:

```
### Choose the Correct Sequential Executor

| Executor Tool Name                  | When Question Mentions ...
| ----------------------------------- | ----------------------------------------------------------------------------
| SequentialBasicinfoExecutor         | basic info, type, name, when others not match
| SequentialGeometryExecutor          | XDim, YDim, depth, thickness, placement, position, location, coordinates
| SequentialQuantityExecutor          | volume, area, width, length, height
| SequentialPropertyExecutor          | room name or number, fire rating, thermal transmittance
| SequentialRelationExecutor          | openings, coverings, assignments, associations
| SequentialSpaceBoundaryExecutor     | room, walls, doors, bounding elements, spatial connections
| SequentialSpatialContainmentExecutor | storeys, spatial structures, contained elements

Use semantic reasoning to pick one executor only.
```

Fig. 8. Prompt template for rule-based executor selection.

```
### Task
1. Analyze the query complexity:
  - Simple query: Single entity type + Single field extraction → Return 1 subtask (unchanged).
  - Compound query: Multiple steps (filter + extract + compute) → Decompose into sequential subtasks.
2. Each subtask must be:
  - Independently executable by GlobalId-level executors.
  - Clear and specific (no ambiguity).
  - Ordered logically (filtering before computation, extraction before aggregation).
3. Preserve dependency order:
  - If subtask B depends on results from subtask A, list A before B.
4. Do NOT invent subtasks that are not implied by the query.
```

Fig. 9. General prompt principles for task decomposition.

result = [key for key, value in data.items() if any(keyword in value.lower() for keyword in ["waste", "soil", "sewer", "sanitary"])].

Consequently, the system identifies 74 drainage pipes with their GlobalIds, which are passed to the next subtask.

- For Subtask 2: Find the diameter of each drainage pipe.

The system first performs schema-guided exploration, and the ObserverAgent summarizes the following toolchain pattern as below:

{"step": 1, "tool": "ExtractPropertySetByGlobalId", "input_schema": {"global_id": "<gid>"}},

{"step": 2, "tool": "QueryWithGlobalIdDict", "input_schema": {"global_id": "<gid>", "field": "PropertySet.Diameter"}}.

The extracted toolchain is applied to all remaining drainage pipe entities through the BulkExecutor, generating structured outputs that associate each GlobalId with its corresponding diameter value, such as:

"1vFudeHNH6kfVLksPjvO1s": "0.082",

"2yZypBm1P0Geeej3iv6nOx": "0.131",

"15Lp1Ca2nEFexULDVEwwSd": "0.131",

"06uxWXUlT4WwN22sA1Epss": "0.131".

In cases of complex, nested field chains (e.g., for thickness), the system performs a similar exploration but generates a toolchain that includes the complete field chain as below:

{"step": 1, "tool": "GetInfoByGlobalId", "input_schema": {"global_id": "<gid>"}},

{"step": 2, "tool": "QueryWithGlobalIdDict", "input_schema": {"global_id": "<gid>", "field": "BasicInfo.Representation.Representations.Items.SweptArea.YDim"}}

- For Subtask 3: Determine the maximum diameter among the drainage pipes.

Once the diameters have been extracted and organized into a GlobalId-Diameter mapping, the system invokes the AggregationAgent to perform the final computation, identifying the maximum diameter from the provided values. The final answer is:

{"answer": "The maximum diameter of the drainage pipes is 0.164, found in the pipe with GlobalId '0n2yQi$YDA28SFAcdqNtR_'."}

5.3. Absolute placement computation and coordinate transformation

In BIM applications, determining an element's absolute placement is essential for geometric reasoning, collision detection, and construction simulation. The IFC-Agent system provides this capability through a schema-guided pipeline that integrates semantic relational tracing and matrix computation. This pipeline leverages both the LLM's reasoning and arithmetic abilities, guided by a structured prompt in Appendix D. It is worth noting that even under Mode I with no AggregationAgent and Python executor available for code generation and execution, the GPT-4o model can still perform numerical operations independently, such as matrix multiplication, to produce accurate coordinate transformation results.

Given an element *IfcBeam* with GlobalId "3tCgZT92j6fw8fXgwCL3Jm" in the FZK-Haus model, the agent computes its world coordinates as follows:

Step A: Build the placement hierarchy.

The agent traces the *ObjectPlacement* field in *BasicInfo* to identify the complete transformation chain. Starting from the beam's *IfcLocalPlacement* (STEP ID #20340), it recursively follows the *PlacementRelTo* references using *GetInfoByStepId* until reaching the root placement where *PlacementRelTo* is None. The complete hierarchy is:

- #20340 → #477 → #432 → #115 (root).

Step B: Construct transformation matrices.

For each IfcLocalPlacement in the hierarchy, the agent extracts the RelativePlacement and constructs the 4 × 4 transformation matrix. In this case:

- IfcLocalPlacement #20340 contains a non-identity transformation with translation (3.570, 4.13, 2.5) and rotation.
- Placements #477, #432, and #115 are identity transformations

Step C: Matrix multiplication and coordinate extraction.

Since the parent placements are identity matrices, the final absolute transformation equals the matrix from #20340. Applying this to the local origin [0,0,0,1] yields the absolute coordinates (3.570, 4.130, 2.500).

The complete reasoning trace is presented in Fig. 10. This case demonstrates that the IFC-Agent can accurately determine an element's absolute placement through schema-guided reasoning and the LLM's intrinsic numerical computation capabilities, even under Mode I without Python executor.

5.4. Query-driven IFC graph construction

The IFC-Agent supports incremental construction of task-specific IFC graphs as a byproduct of query execution. After each query, extracted entities and relationships are cached in two structured dictionaries (*global_id_dict* and *stepid_dict*), which are used to generate a directed, labeled graph representation. Nodes correspond to IFC entities indexed by GlobalId or STEP ID, annotated with parsed properties. Edges encode schema-defined relationships (e.g., *HasAssociations*, *ContainedInStructure*) and STEP-based references (e.g., *MaterialLayers*, *ForLayerSet*).

The graph expands incrementally with each query, progressively

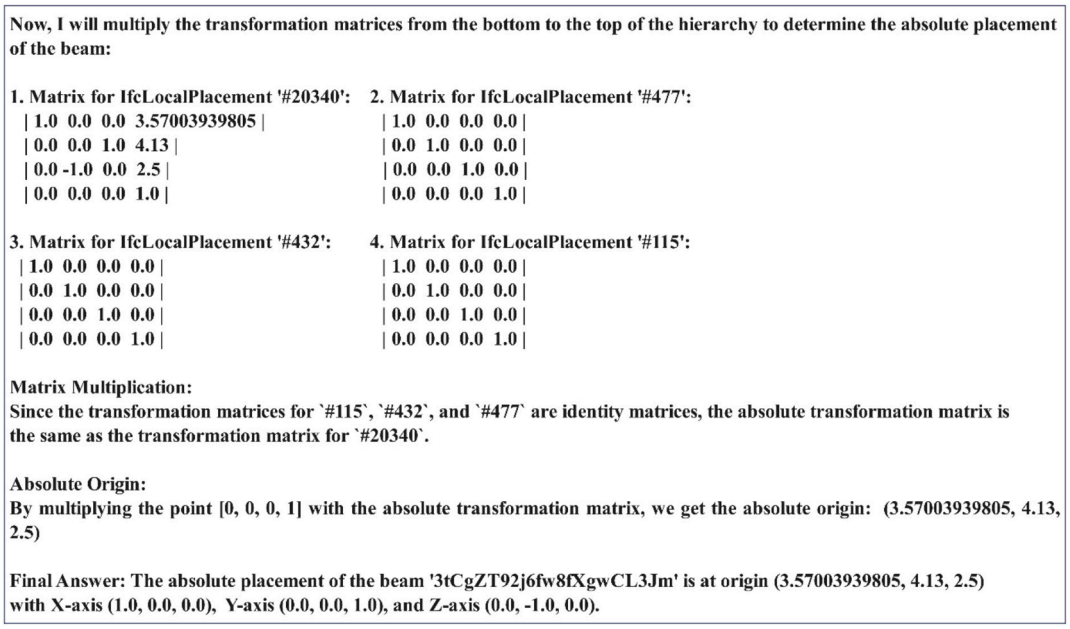


Fig. 10. Reasoning trace for the absolute placement query of an *IfcBeam*.

revealing the explored structure of the IFC model. This enables graph-based analytics such as path visualization, shortest-path search, and connectivity analysis without requiring full model conversion. Fig. 11 illustrates the graph evolution across two consecutive queries:

- Query 1: "What materials are assigned to the slab '1pPHnf7cXCpPsNEnQf8_6B'?"
- Query 2: "What materials are assigned to each slab?"

This query-driven approach addresses a key limitation in BIM analytics: the computational overhead of converting entire IFC models to graph databases. By extracting only task-relevant subgraphs on demand, the system balances analytical flexibility with execution efficiency.

6. Discussion

This section discusses the key findings, comparative analysis, practical implications, and limitations of the proposed framework. Section 6.1 presents experimental insights and a comparison with existing approaches, Section 6.2 discusses practical implications and application scenarios, and Section 6.3 outlines current limitations and directions for future work.

6.1. Key findings and comparative analysis

The benchmark results reveal three critical insights into the framework's design and capabilities, which are discussed alongside a comparison with existing approaches. Section 6.1.1 presents these experimental insights, and Section 6.1.2 provides a qualitative comparison with ifcOWL and GQL4BIM.

6.1.1. Experimental insights

The benchmark experiments reveal three critical insights regarding the IFC-Agent framework's design and capabilities. First, the underlying LLM's reasoning capacity fundamentally determines the system's performance ceiling: GPT-4o achieved 100% success rate, while GPT-3.5-turbo and GPT-4o-mini showed clear degradation with increasing complexity. These results indicate that tool augmentation extends execution capabilities but cannot replace the LLM's intrinsic reasoning capacity. Second, adaptive dual-mode execution is necessary for practical scalability: Mode I provides fine-grained interpretability but fails beyond 20–25 entities, while Mode II enables large-scale execution; neither mode alone is adequate. Third, the experiments demonstrate that LLMs can leverage implicit schema awareness for flexible navigation of IFC structures using atomic tools, and such performance can be enhanced through prompt constraints.

The observed schema awareness operates at two complementary levels. At the first level, when provided with atomic tools but no explicit traversal guidance, the LLM autonomously organizes tool invocations to follow schema-aligned IFC paths. For example, when queried about an element's creation history (without any prompt instructions specifying the traversal sequence), the agent independently identifies the *OwnerHistory* relationship and extracts the owning user (i.e., *IfcPersonAndOrganization*) and the owning application (i.e., *IfcApplication*) references through sequential tool calls, then continues to resolve the *ThePerson*, *TheOrganization*, and *ApplicationDeveloper* entities. Similarly, when extracting geometric properties such as depth or thickness, the agent autonomously constructs multi-step reasoning chains, although the chosen path may not always align with user preferences when multiple valid schema routes exist. This autonomous traversal capability demonstrates schema-aware reasoning and emergent capability beyond simple prompt-based instructions.

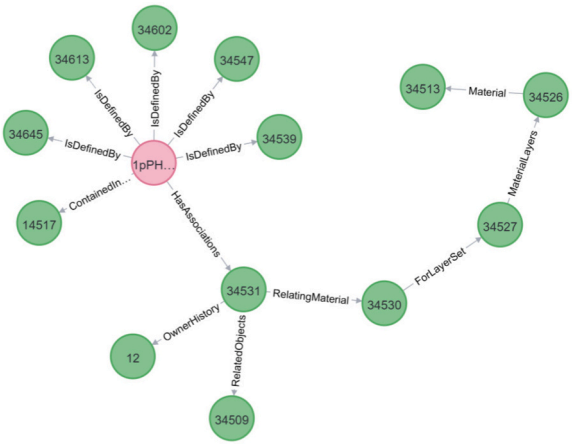
At a second level, when tools return complete STEP structures, advanced LLMs demonstrate schema-aware parsing capabilities that enable efficient entity identification and parameter extraction. For example, when querying thickness from swept profiles, the agent receives the following representations:

- (#15016 = IfcShapeRepresentation(#118,'Body','SweptSolid',(#15,006)), #15024 = IfcShapeRepresentation(#375,'Box','BoundingBox',(#15,023)), #15033 = IfcShapeRepresentation(#15026,'Axis','Curve2D',(#15,031)))

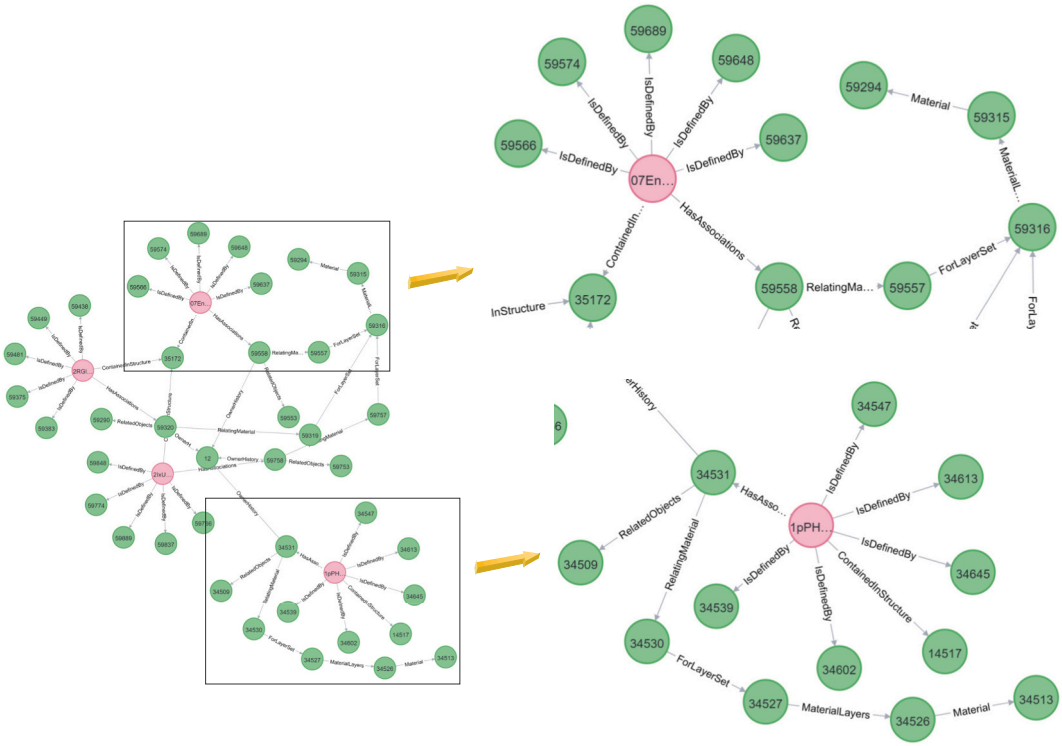
The agent reasons that "the representation with type 'SweptSolid' is likely to contain an IfcExtrudedAreaSolid" and selects #15016 for further traversal.

In another case, upon receiving Items:

- (#15023 = IfcBoundingBox(#15021,4.29,0.24,2.5))



（a） After query 1 (14 nodes, 13 relationships)



（b） After query 2 (45 nodes, 48 relationships)

Fig. 11. Incremental expansion of the IFC graph across two consecutive tasks.

While this less-constrained approach, allowing the agent to observe complete STEP structures, demonstrates strong schema-aware reasoning and improves per-entity efficiency, it leads to step-skipping behaviour that complicates toolchain extraction in Mode II, as the agent may bypass necessary tool invocations and produce execution traces difficult to generalize. Therefore, in Mode II, we constrain tool outputs to return bare field values rather than complete STEP entity expressions, forcing the agent to issue explicit tool calls for each subsequent field access. This produces atomic, step-by-step execution traces from which the ObserverAgent can extract clean, reproducible toolchains that transfer reliably across all entities within homogeneous schema structures.

6.1.2. *Comparison with existing approaches*

IFC-Agent is systematically compared with ifcOWL [14,15] and GQL4BIM [2,3], two representative systems that share the objective of accurate data extraction from IFC models. The functional differences across natural language support, schema navigation, field modification, and extensibility are presented in Table 4. Recent LLM-based BIM systems such as ChatBIM [16] and Graph-RAG approaches [10] have also explored natural language interfaces for BIM data access. However, these systems focus on information retrieval and descriptive aggregation, whereas IFC-Agent addresses execution-intensive operations requiring precise schema-guided reasoning and field-level modifications, which represents a different research scope.

6.2. *Practical implications and applications*

This section discusses how the proposed framework can be applied in practice. Section 6.2.1 describes the prototype implementation, and Section 6.2.2 outlines representative application scenarios and broader impact.

Table 4
Qualitative comparison among IFC-Agent, ifcOWL, and GQL4BIM.

Features	IFC-Agent (this work)	ifcOWL [14,15]	GQL4BIM [2,3]
Natural language input	✔ LLM-based	✖ Not supported	◍ Template-based
STEP-aware tracing	✔ Tool-driven navigation	✔ RDF-based	✔ GQL traversal
Field modification	✔ Editable fields	✖ Query only	✖ Query only
IFC export	✔ Supported	✖ Not supported	✖ Not supported
Memory reuse	✔ Time-stamped logs	✖ Stateless	✖ Stateless
Prompt-based	✔ Yes	✖ Static logic	✖ Static logic
Query-update interface	✔ Yes	✖ Query only	✖ Query only
Tool extensibility	✔ Modular tools	◍ Fixed RDF Rules	◍ Pattern rules
Graph export	✔ CSV (for Neo4j)	✔ RDF native	✔ Neo4j native

✔Fully supported, ✖ Not supported, ◍ Limited supported – the feature is available but restricted to predefined templates or fixed rule-based mechanisms. The specific form of limitation is indicated in each corresponding table cell.

6.2.1. Prototype implementation

To demonstrate the practical viability of the proposed framework, a web-based prototype has been developed using the Gradio library [43], as shown in Fig. 12. The interface provides access to IFC models through conversational queries. The prototype offers two complementary input modes. The system-guided mode, shown in Fig. 12(a), allows users to describe tasks using built-in structured prompts for common operations such as entity retrieval, property extraction, and geometric computation. The prompt-customized mode, illustrated in Fig. 12(b), enables users to input free-form natural language commands and define custom reasoning strategies when specific preferences are required. During execution, the system displays the complete ReAct reasoning trace, including intermediate thoughts, tool invocations, and step-by-step observations, as shown in Fig. 12(c).

6.2.2. Application scenarios and broader impact

The IFC-Agent framework has practical implications for how building information is accessed and manipulated across project workflows. The natural language interface lowers technical barriers that have traditionally limited IFC data access to specialists with programming expertise or users within specific software ecosystems. Domain practitioners, such as architects, engineers, and BIM coordinators, who possess deep domain knowledge but may lack coding skills can now perform schema-guided operations through conversational interaction with native IFC files, without requiring format conversion, query language expertise, or specific software tools. The demonstrated capabilities support application areas that involve iterative exploration of building data. This includes extracting properties across multiple related entities, filtering elements based on parameter combinations, validating attribute values against specified criteria, or modifying fields to generate design alternatives. The conversational interface enables these operations through natural language rather than programming or predefined query templates. While complete domain-specific workflows would require integration with specialized analysis tools, the framework provides the flexible data access layer that such workflows require.

From a broader perspective, this work has implications beyond immediate BIM applications. It demonstrates that LLMs can perform schema-guided reasoning on complex, hierarchical data structures through tool-augmented execution, leveraging implicit schema knowledge without requiring fine-tuning or extensive preprocessing. By enabling natural language interaction with IFC files, this approach can lower barriers to IFC standard adoption, making it accessible to practitioners without requiring specialized technical expertise. The demonstrated capabilities also suggest potential applicability to other technical domains where similar barriers exist between domain experts and structured data, i.e., areas where rich schemas provide powerful abstractions, but technical complexity has limited practical adoption.

6.3. Limitations and future work

The IFC-Agent framework demonstrates effective schema-guided reasoning and tool-augmented execution on IFC data. However, several limitations warrant further investigation. First, while the framework supports field-level modification, it lacks validation mechanisms to verify whether modifications maintain model consistency. Modifying geometric properties may introduce spatial conflicts or violate schema constraints, and changing one attribute may create semantic inconsistencies if related properties are not updated accordingly. Future work should integrate a validation agent to perform post-modification consistency checking, detecting geometric conflicts, schema violations, and semantic dependencies. Second, although the dual-mode execution strategy handles up to 427 entities in the current benchmark, performance on significantly larger models remains untested. Real-world projects often contain tens of thousands of entities, and scalability limits under such conditions are unknown. Future work should evaluate the framework on large-scale, industry-representative models to identify bottlenecks and develop optimization strategies such as hierarchical processing or adaptive caching. Third, the current task decomposition relies on a lightweight, prompt-guided strategy that may not always produce optimal query breakdowns. Future work could explore structured planning approaches, such as alignment-based planners, to enhance decomposition quality and robustness for complex, multi-faceted queries.

Furthermore, the current evaluation is conducted at the query level, and step-level intermediate reasoning traces and tool interactions have not yet been systematically analysed across the benchmark. Such analysis would provide richer evidence of where the framework performs reliably and where failures originate, for instance whether errors arise from incorrect tool selection, misaligned schema traversal, or faulty aggregation. While individual execution traces are illustrated in the case studies, a structured step-level analysis across diverse query types remains an important direction for future work and would contribute to a more complete understanding of the framework's behaviour and remaining limitations.

7. Conclusion

This paper presented IFC-Agent, a modular, schema-guided, and tool-augmented multi-agent framework for natural language interaction with IFC models. The framework establishes a complete workflow from task decomposition to adaptive hybrid execution and result aggregation. It integrates both asynchronous and exploration-aggregation modes to balance flexibility and scalability. In contrast to prior fixed toolchain approaches [3,4,8–10], IFC-Agent performs schema-guided reasoning, where tool selection and chaining are dynamically inferred from the IFC schema rather than predefined procedural templates. During execution, the system employs asynchronous parallel operations for small-scale, heterogeneous tasks (Mode I) and an exploration-aggregation mechanism for large-scale homogeneous queries (Mode II). In the latter case, sampled entities are explored to derive schema-aligned toolchains, which are then observed and generalized by the ObserverAgent for deterministic execution across all remaining entities within a Python environment using LLM-generated codes. This design enables interpretable and scalable reasoning while significantly improving efficiency and robustness through hybrid scheduling and tool-augmented coordination.

A prototype system was developed using LangChain and IfcOpenShell, incorporating both domain-specific tools and schema-aligned prompts, with LLMs as its reasoning core. The framework was

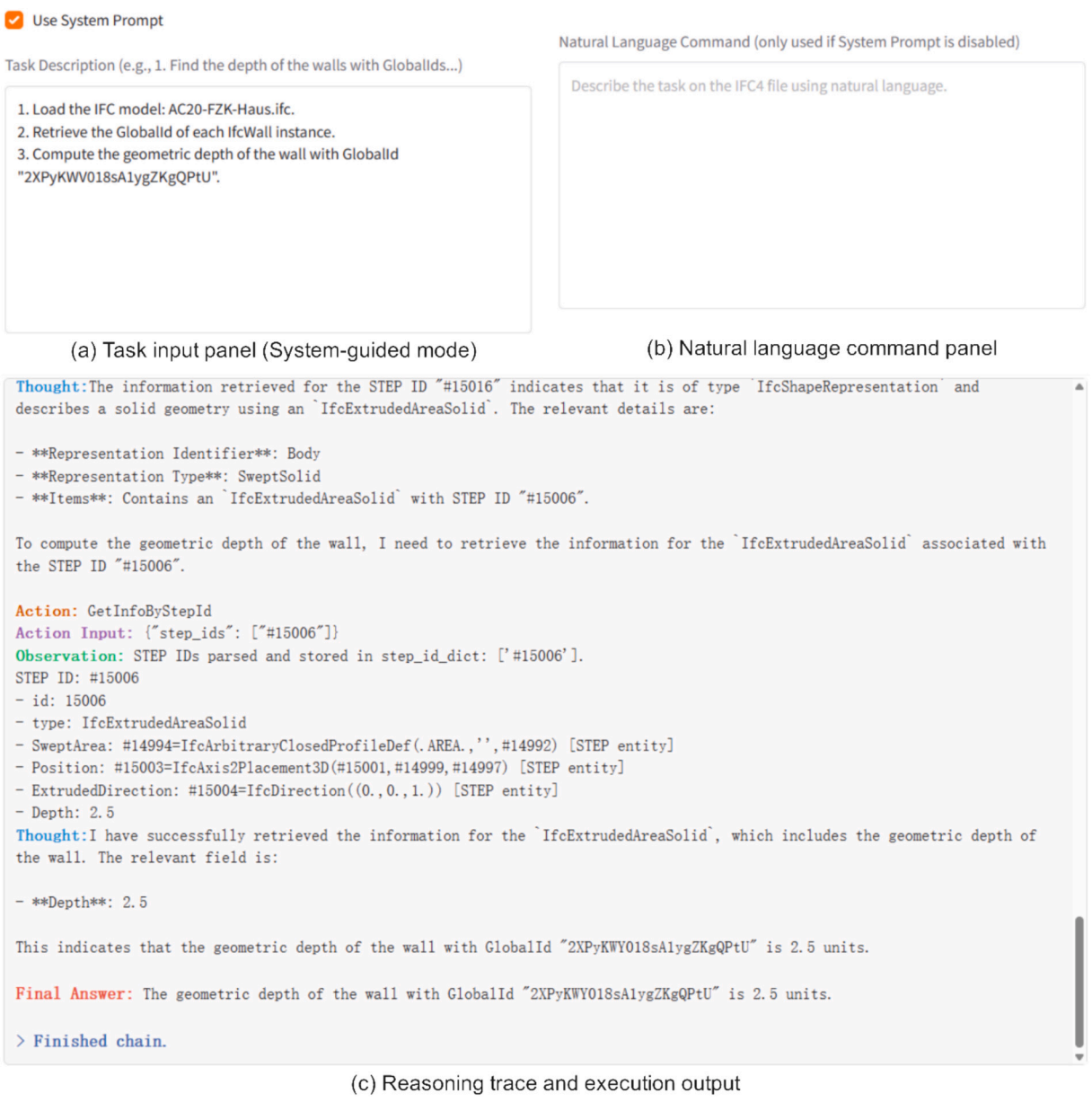


Fig. 12. Web-based prototype interface of the IFC-Agent.

evaluated on a benchmark IFC query set constructed following the taxonomy of prior IFC-query research [3] to ensure standardized assessment. Three OpenAI models, i.e., GPT-3.5-turbo, GPT-4o-mini, and GPT-4o, were compared under identical configurations, and GPT-4o successfully completed all benchmark tasks, demonstrating strong multi-hop reasoning capability, execution efficiency, and robustness. These capabilities were further validated through four representative case studies, which collectively confirmed the framework's reliability and versatility: (i) field-level querying and modification verified schema-guided reasoning and rule-based executor selection for precise field access; (ii) task decomposition and exploration-aggregation execution proved the robustness of sampled exploration and LLM-generated code aggregation for large-scale entity reasoning; (iii) coordinate transformation validated the agent's multi-hop relational reasoning and intrinsic numerical capability to perform matrix computation without external code execution; (iv) incremental IFC-graph construction demonstrated the framework's ability to accumulate semantic knowledge and progressively expand its understanding of the model through each query.

The IFC-Agent framework demonstrates effective schema-guided reasoning and tool-augmented execution on IFC data through a modular multi-agent design. The current implementation achieves scalable dual-mode execution and interpretable agent coordination. Future work should address several areas for enhancement. First, while the framework supports field-level modification, integrating validation mechanisms to verify model consistency after modifications would improve reliability. Second, although the dual-mode execution strategy handles up to 427 entities in the current benchmark, evaluation on larger-scale, industry-representative models is needed to fully assess scalability limits and identify optimization opportunities. Third, the current task decomposition mechanism relies on a lightweight, prompt-guided strategy that could benefit from structured planning approaches to enhance decomposition quality for complex, multi-faceted queries. Addressing these areas will strengthen the framework's practical applicability and robustness for real-world BIM workflows.

CRediT authorship contribution statement

Yan Gao: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Investigation, Data curation, Conceptualization. **Fuji Hu:** Formal analysis, Data curation. **Chengzhang Chai:** Validation, Software. **Yiwei Weng:** Supervision, Project administration, Funding acquisition. **Haijiang Li:** Resources, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

The authors would like to gratefully acknowledge the support provided by The Hong Kong Polytechnic University (P0058004). The authors also thank the editors and reviewers for their helpful comments.

Appendix A. Benchmark question set for IFC-agent

ID	Query	Traversal Depth	Execution Complexity	Aims	Results
1	Get the basic information of an IFC element / entity by ID (i.e., GlobalId or STEP ID).	0 hop	Simple Retrieval	Retrieve, List	Y/Y/Y
2	Find IFC elements by simple attribute matching (e.g., Name, Tag).	0 hop	Simple Retrieval	Retrieve, List	Y/Y/Y
3	Retrieve all instances of a given IFC type (e.g., all IfcDoors).	0 hop	Simple Retrieval	Retrieve, List	Y/Y/Y
4	List all property or quantity attributes associated with an IFC element.	0 hop	Simple Retrieval	Retrieve, List	Y/Y/Y
5	List all relationship entities associated with an IFC element (including spatial boundaries and spatial containment), without traversal or expansion.	0 hop	Simple Retrieval	Retrieve, List	Y/Y/Y
6	Get all instances of a given type that satisfy a numeric attribute condition (e.g., width $>$ 0.9 m).	1–2 hop	Relational Navigation	Retrieve, Traverse, Filter	N/Y/Y
7	Retrieve the complete spatial structure hierarchy (site, building, storey, space).	3+ hop	Multi-hop Reasoning	Retrieve, Traverse, List	N/Y/Y
8	Find all building elements contained in a specific spatial structure element (e.g., storey 2).	3+ hop	Multi-hop Reasoning	Retrieve, Extract, Traverse, List	N/Y/Y
9	Get all spaces and their bounding building elements (e.g., walls, doors).	2–3 hop	Multi-hop Reasoning	Retrieve, Extract, Traverse, List	N/Y/Y
10	Retrieve spaces that are connected by doors or virtual elements (space accessibility).	2–3 hop	Multi-hop Reasoning	Retrieve, Extract, Compare, Filter	N/N/Y
11	Find walls that are connected to other walls.	2–3 hop	Multi-hop Reasoning	Retrieve, Extract, Traverse, List	N/Y/Y
12	Retrieve all wall elements and their associated property sets and quantity sets.	1–2 hop	Relational Navigation	Retrieve, Extract, List	N/Y/Y
13	Get the materials assigned to each building element via IfcRelAssociatesMaterial.	2–3 hop	Multi-hop Reasoning	Retrieve, Extract, Traverse, List	N/Y/Y
14	Retrieve the local placement of a specific element and extract its local position coordinates.	1–2 hop	Relational Navigation	Retrieve, Traverse, Identify	N/Y/Y
15	Trace the relative placement chain of a given element.	2–3 hop	Multi-hop Reasoning	Retrieve, Traverse, Trace	N/Y/Y
16	Get the absolute (world) placement of a specific element.	3+ hop	Execution-intensive Queries	Retrieve, Traverse, Calculate	N/Y/Y
17	Retrieve the representation type and representation items (e.g., swept solids, BRep).	2–3 hop	Multi-hop Reasoning	Retrieve, Traverse	N/Y/Y
18	Extract geometry values such as Depth, Height, or Length from IfcExtrudedAreaSolid.	2–3 hop	Multi-hop Reasoning	Retrieve, Traverse	N/N/Y
19	Tell me the complete creation history of a specific element, including who created it, which organization, what software was used, and when it was created.	2–3 hop	Relational Navigation	Retrieve, Traverse	N/Y/Y
20	Retrieve the classification reference for each element (e.g., Omniclass code).	2–3 hop	Multi-hop Reasoning	Retrieve, Extract, Traverse, List	N/Y/Y
21	Get the openings hosted by each wall (HasOpenings).	2–3 hop	Multi-hop Reasoning	Retrieve, Extract, Traverse, List	N/Y/Y
22	Find the doors or windows that fill the opening (FillsVoids).	2–3 hop	Multi-hop Reasoning	Retrieve, Extract, Traverse, List	N/Y/Y
23	Trace a relation and identify its Relating and Related entities.	2–3 hop	Multi-hop Reasoning	Extract, Traverse, List	N/Y/Y
24	Explore the IfcRelDefinesByProperties relationship for a specific element and show what it contain.	1–2 hop	Relational Navigation	Extract, Traverse	N/Y/Y
25	How many individual quantity items (e.g., IfcQuantityLength, IfcQuantityArea) are defined for each wall or slab?	1–2 hop	Relational Navigation	Retrieve, Extract, Aggregate	N/N/Y
26	Perform recursive traversal of spatial containment.	2–3 hop	Execution-intensive Queries	Extract, Traverse, Aggregate	N/Y/Y

(*continued on next page*)

(*continued*)

ID	Query	Traversal Depth	Execution Complexity	Aims	Results
27	Traverse geometry reference chains of a specific element.	3+ hop	Execution-intensive Queries	Retrieve, Traverse	N/Y/Y
28	Trace the full property definition chain of a specific beam.	3+ hop	Multi-hop Reasoning	Retrieve, Traverse, Trace	N/Y/Y
29	Which doors have thermal transmittance >0.2.	1–2 hop	Relational Navigation	Retrieve, Extract, Filter	N/Y/Y
30	Find external walls with thickness > 0.25 m.	2–3 hop	Constraint Filtering	Retrieve, Extract, Filter	N/Y/Y
31	Find all external walls with thermal transmittance >1 and have thickness > 0.25 m.	3+ hop	Constraint Filtering	Retrieve, Extract, Filter	N/Y/Y
32	Find doors that connect two rooms of different storeys.	3+ hop	Constraint Filtering	Retrieve, Extract, Traverse, Compare, Filter	N/N/Y
33	Detect isolated rooms (no connectivity to other rooms).	2–3 hop	Multi-hop Reasoning	Retrieve, Extract, Compare, Classify, Filter	N/Y/Y
34	Identify overlapping rooms (rooms sharing multiple common boundaries).	2–3 hop	Multi-hop Reasoning	Retrieve, Extract, Compare, Filter	N/N/Y
35	Identify all spaces directly connected to the exterior.	2–3 hop	Multi-hop Reasoning	Retrieve, Extract, Classify, Filter	N/N/Y
36	Which slabs are connected to columns?	2–3 hop	Relational Navigation	Retrieve, Extract, Traverse, List	N/Y/Y
37	Identify all space elements associated with a Uniclass classification code ‘000’.	1–2 hop	Relational Navigation	Extract, Traverse, List	N/N/Y
38	Retrieve rooms that share a wall with bathrooms.	2–3 hop	Multi-hop Reasoning	Retrieve, Extract, Compare, Filter	N/Y/Y
39	Get all load-bearing walls that span two or more storeys.	2–3 hop	Constraint Filtering	Retrieve, Extract, Compare, Filter	N/N/Y
40	Count how many openings exist per storey.	1–2 hop	Multi-hop Reasoning	Retrieve, Extract, Aggregate	N/N/Y
41	How many staircases are there in the building, and do they all have the same height?	2–3 hop	Multi-hop Reasoning	Retrieve, Traverse, Compare	N/Y/Y
42	Find the depth of each beam and identify the beam with the maximum depth.	3+ hop	Multi-hop Reasoning	Retrieve, Traverse, Aggregate	N/N/Y
43	Group doors by OverallHeight, and report the count and instances per height category.	1–2 hop	Relational Navigation	Retrieve, Traverse, Aggregate	N/Y/Y
44	Compute the gross area (OverallHeight × OverallWidth) for each door and the total gross door area.	1–2 hop	Relational Navigation	Retrieve, Traverse, Calculate, Aggregate	N/N/Y
45	Group windows by OverallWidth and report the count and instances per width category.	1–2 hop	Relational Navigation	Retrieve, Traverse, Aggregate	N/Y/Y
46	Group walls by thickness derived from the swept profile.	3+ hop	Multi-hop Reasoning	Retrieve, Traverse, Aggregate	N/N/Y
47	Find the material of a given wall and change its material layer thickness to 0.3. Then, export the updated IFC file.	3+ hop	Execution-intensive Queries	Retrieve, Traverse, Modify, Export	N/N/Y
48	Starting from entrance door, find all reachable rooms through internal doors and report isolated rooms.	2–3 hop	Execution-intensive Queries	Retrieve, Extract, Traverse, List	N/N/Y
49	Find all windows located on a specific storey, with an overall width of approximately 1.0 m, and represented using IfcFacetedBrep.	1–2 hop	Constraint Filtering	Retrieve, Traverse, Filter, Extract	N/N/Y
50	Find all exterior walls on the ground floor that have thickness between 0.25 m and 0.40 m, and contain at least one window opening. List their GlobalIds and window counts.	2–3 hop	Constraint Filtering	Retrieve, Traverse, Filter, Extract	N/N/Y
51	Find all exterior walls on the ground floor, update their doors' FireRating property to ‘90-min’, and export the new IFC model.	2–3 hop	Execution-intensive Queries	Retrieve, Extract, Traverse, Filter, Modify, Export	N/N/Y
52	Find all beams and slabs on the upper floor, update their beams' material name to ‘Eiche’ and their slabs' ThermalTransmittance property to 0.35, and export the new IFC model.	2–3 hop	Execution-intensive Queries	Retrieve, Extract, Traverse, Modify, Export	N/N/Y
53	What is the capacity of the hot water boiler?	1–2 hop	Relational Navigation	Retrieve, Filter, Traverse, List	N/Y/Y
54	Group flow terminal devices by type and report the count and IDs for each type.	1–2 hop	Relational Navigation	Retrieve, Traverse, Aggregate	N/Y/Y
55	What is the total heating capacity of all radiators in the building?	1–2 hop	Relational Navigation	Retrieve, Filter, Extract, Aggregate	N/Y/Y
56	For all HVAC ducts, compute the circumference as π × diameter, and identify the duct with the maximum circumference together with its GlobalId.	1–2 hop	Execution-intensive Queries	Retrieve, Traverse, Filter, Extract, Calculate	N/N/Y
57	What is the total length of all the supply pipes in the building?	2–3 hop	Multi-hop Reasoning	Retrieve, Filter, Traverse, Aggregate	N/N/Y
58	What is the maximum diameter of the drainage pipes?	2–3 hop	Multi-hop Reasoning	Retrieve, Filter, Traverse, Aggregate	N/N/Y
59	Get all components controlled by a specific control element (e.g., schedule).	3+ hop	Execution-intensive Queries	Retrieve, Extract, Traverse, Filter, List	N/N/Y
60	Retrieve all construction tasks assigned to elements on a specific storey.	3+ hop	Execution-intensive Queries	Retrieve, Extract, Compare, List	N/N/Y

Y – Success; N – Unsuccess in the results under order GPT-3.5 / GPT-4o-mini / GPT-4o.

Appendix B. Ablation study on execution strategy

ID	Questions / Tasks	Entity Type	Number	Task Type	Mode I	Mode II (Recovery)
1	How many staircases are there in this building, and do they all have the same height?	Stairs	2	Counting + Comparison	Success	NA
2	Find the depth of each beam and identify the beam with the maximum depth.	Beams	8	Per-entity Value Extraction + Max selection	Success	NA
3	Group doors by OverallHeight and report the count and instances per height category.	Doors	14	Grouping + Counting + Listing	Success	NA
4	Compute the gross area (OverallHeight × OverallWidth) for each door and the total gross door area.	Doors	14	Per-entity Computation + Sum Aggregation	Success	NA
5	Group windows by OverallWidth and report the count and instances per width category.	Windows	24	Grouping + Counting + Listing	Success	NA
6	Compute the gross area (OverallHeight × OverallWidth) for each window and the total gross window area.	Windows	24	Per-entity Computation + Sum Aggregation	Wrong	Success
7	Group walls by thickness derived from the swept profile.	Walls	56	Geometry-derived Attribute Extraction + Grouping	Failure	Success
8	Group flow terminal devices by type and report the count and identifiers for each type.	Flow Terminals	105	Grouping + Counting + Listing	Failure	Success
9	Find HAVC ducts from IfcSegment.	Flow Segments	427	Retrieval + Filtering	Failure	Success

Success – task completed correctly;
Wrong – task completed but with incorrect results (e.g., missing entities or misaligned intermediate mappings);
Failure – task not fully completed according to the specification.

Appendix C. IfcWall material modification

Step 1: Trace material association
The agent begins by calling *ExtractRelationsByGlobalId* to retrieve the relation *HasAssociations* for the wall 3PfS__Y_DBAfq5naM6zD2Z, which yields:
- *HasAssociations*: [#17045] → STEP entity: *IfcRelAssociatesMaterial*

Step 2: Resolve material relation
Using *GetInfoByStepId* and *QueryWithStepIdDict* on #17045, the agent identifies:
- *RelatingMaterial*: #17044 → *IfcMaterialLayerSetUsage*

Step 3: Follow to Layer Set
The agent resolves #17044, revealing:
- *ForLayerSet*: #15069 → *IfcMaterialLayerSet*

Step 4: Find material layer
Parsing #15069 returns the field *IfcMaterialLayer* #15067 with:
- *Material*: #15046 = *IfcMaterial*('Leichtbeton 102890359')
- *LayerThickness*: 0.24

✔ Find the wall material successfully.

Step 5: Modify layer thickness
To update the wall's thickness, the agent performs:
SetFieldByStepId:

```
{
 "step_id": "#15067",
 "field": "LayerThickness",
 "value": 0.3
}
```

✔ The field is updated successfully in the IFC model.

Step 6: Export modified model
The agent calls:
ExportModifiedIFC:

```
{
 "output_path": "export/updated_model.ifc"
}
```

✔ A new IFC file is generated with the updated layer thickness.

Appendix D. Prompt for absolute placement computation

```
If the question asks about world or absolute placement:

Step A: Build the placement hierarchy
- Start from ObjectPlacement in BasicInfo → get the IfcLocalPlacement STEP ID
- Use GetInfoByStepId to get its details (PlacementRelTo, RelativePlacement)
- If PlacementRelTo exists, recursively follow the chain until PlacementRelTo = None
- Record the hierarchy: [root_placement_id, parent_placement_id, ..., current_placement_id]

Step B: Collect all transformation data
For each IfcLocalPlacement in the hierarchy (from root to current):
- Use GetInfoByStepId to get the RelativePlacement (IfcAxis2Placement3D) STEP ID
- Use GetInfoByStepId to get Location, Axis, RefDirection references
- Call ComputePlacementMatrixFromStepId to get the 4×4 transformation matrix

Step C: Matrix multiplication
After collecting all matrices [M0, M1, M2, ..., Mn] from root to current:

1. Multiply matrices step by step:
  - Result = M0 (start with root matrix)
  - Result = Result × M1
  - Result = Result × M2
  - ... continue until all matrices are multiplied
  - Final_Matrix = M0 × M1 × M2 × ... × Mn

2. Apply to origin point:
  - Point = [0, 0, 0, 1] (homogeneous coordinates)
  - Absolute_Position = Final_Matrix × Point
  - Extract [x, y, z] from the result

3. Show your work:
  - Display each matrix clearly
  - Show the step-by-step multiplication process
  - Verify the final coordinates

CRITICAL REQUIREMENTS:
ALWAYS call ComputePlacementMatrixFromStepId for each IfcLocalPlacement in the hierarchy
ALWAYS show the actual matrix multiplication calculations step by step
ALWAYS multiply ALL matrices in the chain, don't skip any
ALWAYS extract coordinates from the final matrix-vector multiplication
Do NOT assume any matrix is identity without verification
Do NOT skip intermediate matrices
Do NOT just use the last matrix's origin as the absolute position
```

Data availability

Data will be made available on request.

References

[1] Industry Foundation Classes (IFC) - buildingSMART International, Accessed: Oct. 19, 2025. [Online]. Available: https://www.buildingsmart.org/standards/bsi-standards/industry-foundation-classes/.

[2] J. Zhu, P. Wu, X. Lei, IFC-graph for facilitating building information access and query, Autom. Constr. 148 (2023), https://doi.org/10.1016/j.autcon.2023.104778.

[3] J. Zhu, N. Nisbet, M. Yin, R. Wei, I. Brilakis, Releasing the power of graph for building information discovery, Autom. Constr. 172 (2025), https://doi.org/10.1016/j.autcon.2025.106034.
[4] S. Hellin, S. Nousias, A. Borrmann, Natural language information retrieval from BIM models: An LLM-based agentic workflow approach, in: Proceedings of the 2025 European Conference on Computing in Construction (EC3 2025), Porto, Portugal, 2025, https://doi.org/10.35490/EC3.2025.265.
[5] D.H. Hagos, R. Battle, D.B. Rawat, 'Recent Advances in Generative AI and Large Language Models: Current Status, Challenges, and Perspectives, Available: http://arxiv.org/abs/2407.14962, 2024.
[6] C. Du, S. Nousias, A. Borrmann, in: Towards a copilot in BIM authoring tool using a large language model-based agent for intelligent human-machine interaction, University of Vigo, Vigo, Spain, 2024, pp. 403–412, in: https://doi.org/10.35869/Proceedings_EGICE2024.
[7] C. Du, S. Esser, S. Nousias, A. Borrmann, Text2BIM: Generating Building Models Using a Large Language Model-based Multi-Agent Framework, Available: http://arxiv.org/abs/2408.08054, 2024.
[8] L. Zhang, X. Fu, Y. Li, J. Chen, Large language model-based agent Schema and library for automated building energy analysis and modeling, Autom. Constr. 176 (2025), https://doi.org/10.1016/j.autcon.2025.106244.
[9] P. Guo, H. Xue, J. Ma, J.C.P. Cheng, Advancing BIM information retrieval with an LLM-based query-domain-specific language and library code function alignment system, Autom. Constr. 178 (2025), https://doi.org/10.1016/j.autcon.2025.106374.
[10] S. Iranmanesh, H. Saadany, E. Vakaj, 'LLM-assisted graph-RAG information extraction from IFC data, in: Proceedings of the 2025 European Conference on Computing in Construction (EC3 2025), Porto, Portugal, 2025, https://doi.org/10.35490/EC3.2025.366.
[11] S. Du, Y. Gao, W. Tong, Y. Weng, Towards agent: LLM-based framework for robotic construction by leveraging IFC, in: Proceedings of the 42nd International Symposium on Automation and Robotics in Construction (ISARC 2025), IAARC Publications, Montreal, Canada, 2025, pp. 358–365, https://doi.org/10.22260/ISARC2025/0048.
[12] GitHub - langchain-ai/langchain: Build Context-aware Reasoning Applications, Accessed: Oct. 19, 2025. [Online]. Available: https://github.com/langchain-ai/langchain.
[13] GitHub - IfcOpenShell/IfcOpenShell: Open Source IFC Library and Geometry Engine, Accessed: Oct. 19, 2025. [Online]. Available: https://github.com/IfcOpenShell/IfcOpenShell.
[14] P. Pauwels, W. Terkaj, EXPRESS to OWL for construction industry: towards a recommendable and usable ifcOWL ontology, Autom. Constr. 63 (2016) 100–133, https://doi.org/10.1016/j.autcon.2015.12.003.
[15] P. Pauwels, T. Krijnen, W. Terkaj, J. Beetz, Enhancing the ifcOWL ontology with an alternative representation for geometric data, Autom. Constr. 80 (2017) 77–94, https://doi.org/10.1016/j.autcon.2017.03.001.
[16] J.-R. Lin, P. Pan, ChatBIM: Interactive data retrieval and visualization from BIM, in: Proceedings of the 10th International Conference on Innovative Production and Construction (IPC 2024), Perth, Australia, 2024, pp. 265–266, https://doi.org/10.13140/RG.2.2.31563.20006.
[17] C. Du, Z. Deng, S. Nousias, A. Borrmann, Towards commands recommender system in BIM authoring tool using Transformers, in: Proceedings of the 31st International Workshop on Intelligent Computing in Engineering (EG-ICE 2024), Vigo, Spain, 2024. Available: https://arxiv.org/abs/2406.10237.
[18] K. Forth, A. Borrmann, Semantic enrichment for BIM-based building energy performance simulations using semantic textual similarity and fine-tuning multilingual LLM, J. Build. Eng. 95 (2024), https://doi.org/10.1016/j.jobe.2024.110312.
[19] Y. Talebirad, A. Nadiri, Multi-Agent Collaboration: Harnessing the Power of Intelligent LLM Agents, Available: http://arxiv.org/abs/2306.03314, 2023.
[20] S. Yuan, K. Song, J. Chen, X. Tan, Y. Shen, K. Ren, et al., EASYTOOL: Enhancing LLM-based agents with concise tool instruction, in: Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics (NAACL 2025), Albuquerque, USA, 2025, pp. 951–972, https://doi.org/10.18653/v1/2025.naacl-long.44.
[21] M. Li, Y. Zhao, B. Yu, F. Song, H. Li, H. Yu, et al., API-Bank: A comprehensive benchmark for tool-augmented LLMs, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023), Singapore, 2023, pp. 3102–3116, https://doi.org/10.18653/v1/2023.emnlp-main.187.
[22] X. Li, S. Wang, S. Zeng, Y. Wu, Y. Yang, A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges, Vicinagearth 1 (1) (2024) 9, https://doi.org/10.1007/s44336-024-00009-2.
[23] C. Du, Z. Deng, S. Nousias, A. Borrmann, Predictive modeling: BIM command recommendation based on large-scale usage logs, Adv. Eng. Inform. 68 (2025), https://doi.org/10.1016/j.aei.2025.103574.
[24] crewAIInc/crewAI: Framework for Orchestrating Role-playing, Autonomous AI Agents. By Fostering Collaborative Intelligence, CrewAI Empowers Agents to Work together Seamlessly, Tackling Complex Tasks, Accessed: Jun. 08, 2025. [Online]. Available: https://github.com/crewAIInc/crewAI.
[25] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, et al., ChatDev: Communicative agents for software development, in: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024), Bangkok, Thailand, 2024, pp. 15174–15186, https://doi.org/10.18653/v1/2024.acl-long.810.
[26] Y. Deng, J.C.P. Cheng, C. Anumba, Mapping between BIM and 3D GIS in different levels of detail using schema mediation and instance comparison, Autom. Constr. 67 (2016) 1–21, https://doi.org/10.1016/j.autcon.2016.03.006.
[27] T.W. Kang, C.H. Hong, IFC-CityGML LOD mapping automation using multiprocessing-based screen-buffer scanning including mapping rule, KSCE J. Civ. Eng. 22 (2) (2018) 373–383, https://doi.org/10.1007/s12205-017-0595-9.
[28] ChatGPT - IFC Insight, Accessed: Oct. 04, 2025. [Online]. Available: https://chatgpt.com/g/g-gRmWwxXkk-ifc-insight?model=gpt-4o.
[29] IfcOpenShell Contributors, IfcOpenShell: Open Source IFC Library and Geometry Engine, Available: https://github.com/IfcOpenShell/IfcOpenShell, 2025.
[30] F. Gerold, IfcPlusPlus: Open Source C++ IFC Implementation, Accessed: Oct. 04, 2025. [Online]. Available: https://github.com/ifcquery/ifcplusplus.
[31] T.O. Company, IFC.js: A JavaScript Toolkit for BIM, Accessed: Oct. 04, 2025. [Online]. Available: https://github.com/ThatOpen/web-ifc-viewer.
[32] J. Beetz, L. van Berlo, R. de Laat, P. van den Helm, Bimserver.org – an Open Source IFC Model Server, in: Proceedings of the 27th International Conference on Applications of IT in the AEC Industry (CIB W78), Cairo, Egypt, 2010. Available: https://itc.scix.net/pdfs/w78-2010-51.pdf.
[33] BlenderBIM Add-on for Building and Exporting an IFC Model - Wiki.OSArch, Accessed: Jul. 26, 2025. [Online]. Available: https://wiki.osarch.org/index.php?title=BlenderBIM_Add-on/BlenderBIM_Add-on_for_building_and_exporting_an_IFC_model.
[34] FreeCAD - Wikipedia, Accessed: Jul. 26, 2025. [Online]. Available: https://en.wikipedia.org/wiki/FreeCAD.
[35] buildingSMART International, Industry Foundation Classes (IFC) 4.0.2.1 – Addendum 2 Technical Corrigendum 1, Accessed: Oct. 04, 2025. [Online]. Available: https://standards.buildingsmart.org/IFC/RELEASE/IFC4/ADD2/HTML/.
[36] T. Krijnen, J. Beetz, A SPARQL query engine for binary-formatted IFC building models, Autom. Constr. 95 (2018) 46–63, https://doi.org/10.1016/j.autcon.2018.07.014.
[37] Y. Gao, G. Xiong, H. Li, J. Richards, Exploring bridge maintenance knowledge graph by leveraging GrapshSAGE and text encoding, Autom. Constr. 166 (2024), https://doi.org/10.1016/j.autcon.2024.105634.
[38] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, et al., ReAct: Synergizing reasoning and acting in language models, in: Proceedings of the 11th International Conference on Learning Representations (ICLR 2023), Kigali, Rwanda, 2023. Available: https://openreview.net/forum?id=WE_vluYUL-X.
[39] KIT IFC Examples, Available: https://www.ifcwiki.org/index.php?title=KIT_IFC_Examples.
[40] jakob-beetz/DataSetSchependomlaan, Accessed: Jul. 26, 2025. Available: https://github.com/jakob-beetz/DataSetSchependomlaan.
[41] ifc-bench/projects/duplex at main · sylvainHellin/ifc-bench · GitHub, Accessed: Oct. 13, 2025. Available: https://github.com/sylvainHellin/ifc-bench/tree/main/projects/duplex.
[42] Free IFC viewer online|usBIM|ACCA software, Accessed: Dec. 28, 2025. Available: https://www.accasoftware.com/en/ifc-viewer-on-line.
[43] 'Gradio, Accessed: Oct. 18, 2025. Available: https://www.gradio.app/.